PROJECT 03

The Power of Flow Models

Overview

This project is about flow matching, a way to generate images by moving step by step from Gaussian noise to a clean image. Part A uses the pretrained text-to-image model PixNerd-XXL-P16-T2I (512×512) without any training: it implements the noising process, one-step and iterative denoising, classifier-free guidance, and uses them for image editing, visual anagrams and hybrid images. Part B builds and trains a small UNet from scratch on MNIST, first as a one-step denoiser and then as a time- and class-conditioned flow matching model.

Both parts use the same convention: `t = 0` is pure noise and `t = 1` is a clean image. The model predicts the velocity $x_1 - x_0$, not the noise.

Part A: Exploring a pretrained flow model

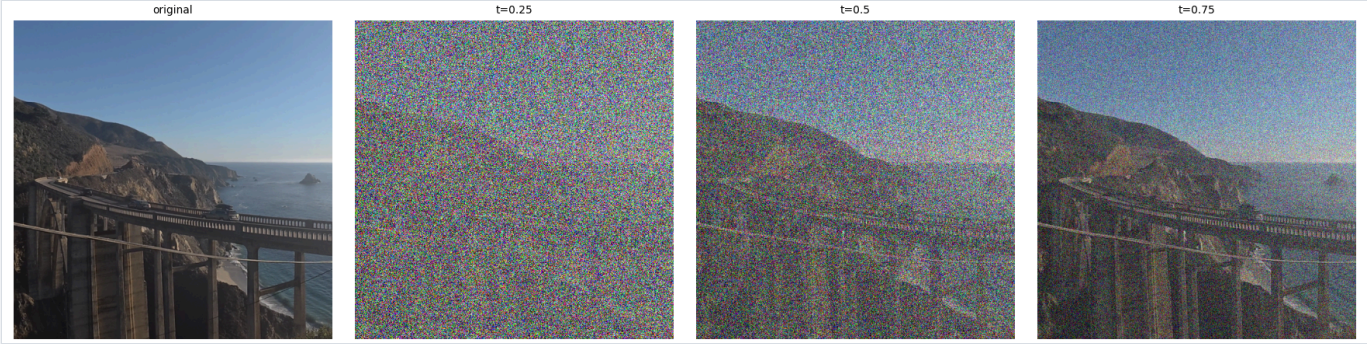
A.1.1: Constructing noisy training inputs

A noisy input is a linear interpolation between Gaussian noise ε and the clean image x_1:

$$x_t = (1 - t)\,\varepsilon + t\,x_1$$

This does not just add noise, it also scales the image down by t. One noise tensor ε is sampled and reused for all three times, so the images only differ in the mixing weight. My own image is a photo of the Bixby Creek Bridge in Big Sur, center-cropped to 512×512.

```
def interpolate(image, noise, t):
    return (1 - t) * noise + t * image
```

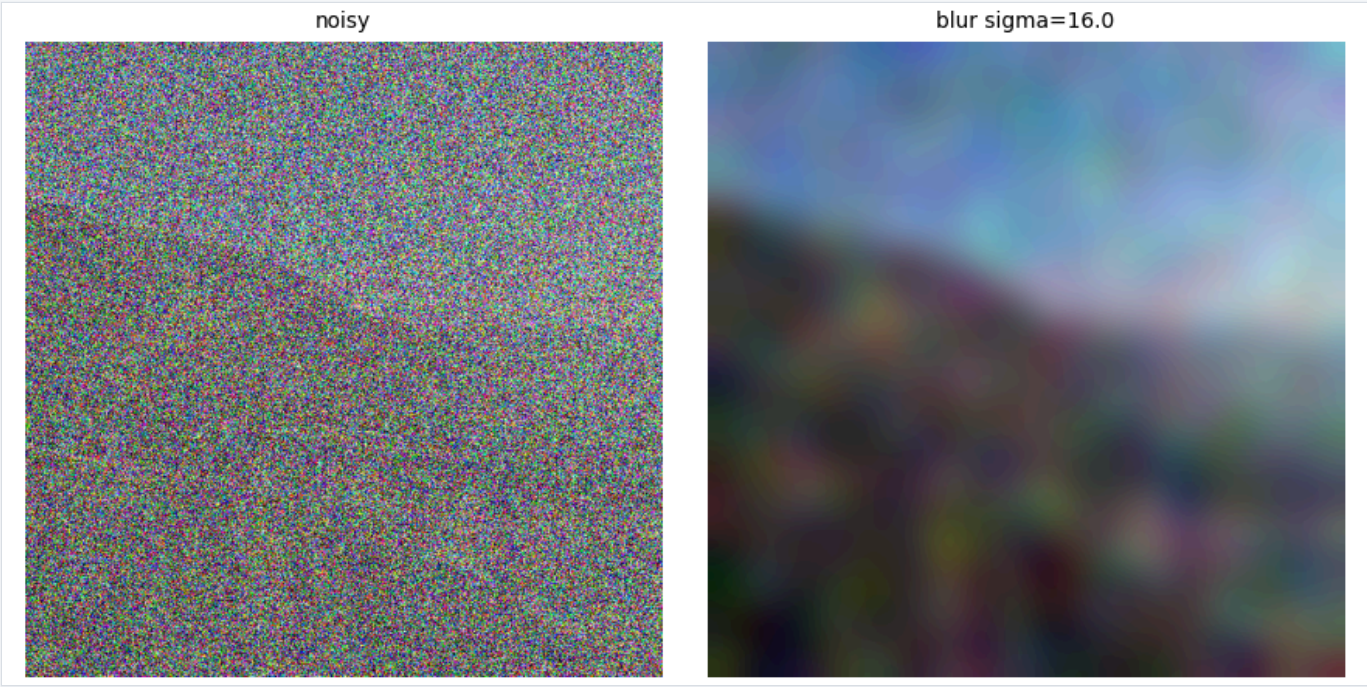


original and noisy inputs at t = 0.25, 0.50, 0.75

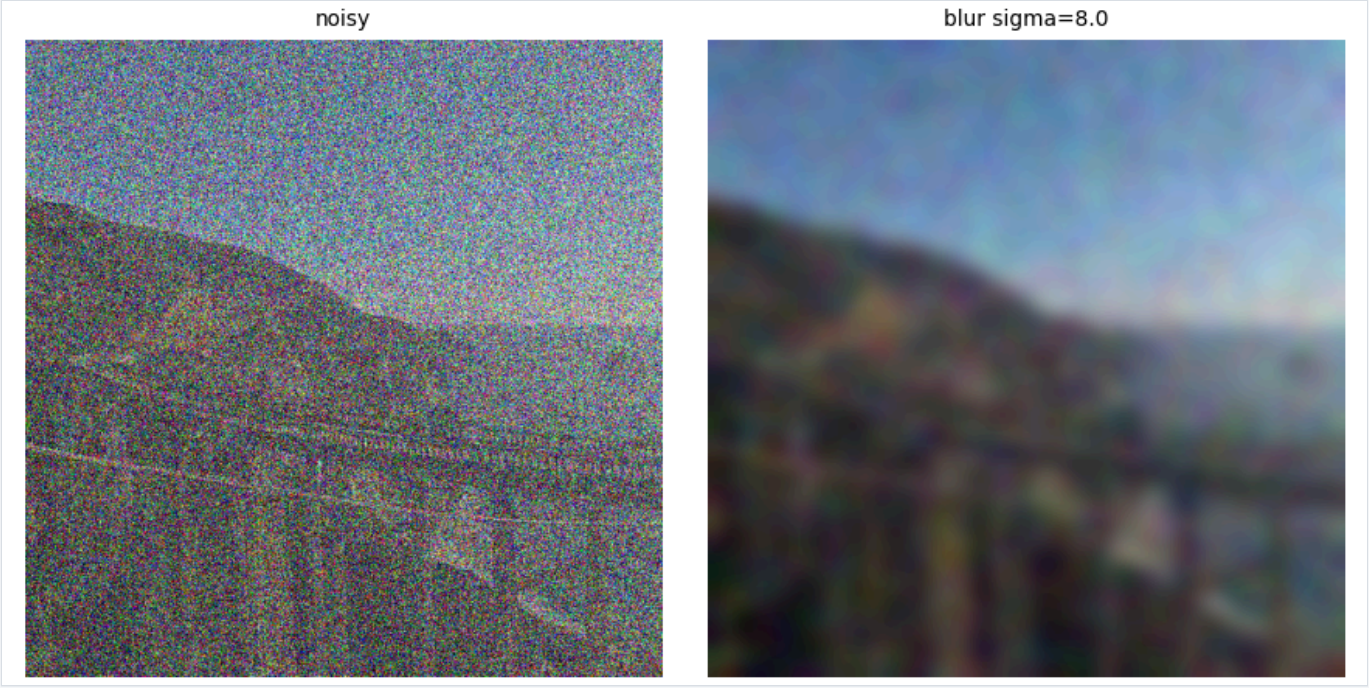
The smaller t, the more noise: at t = 0.25 only the rough layout of the bridge and the sky is visible, at t = 0.75 the image is clearly recognizable.

A.1.2: Classical denoising

As a baseline, the noisy image is first divided by t to restore the image's amplitude, then blurred with a Gaussian. For each t, several σ between 1 and 16 were tried and the one closest to the original (lowest MSE) is shown.



t = 0.25: noisy input and best Gaussian blur



t = 0.50

t = 0.75

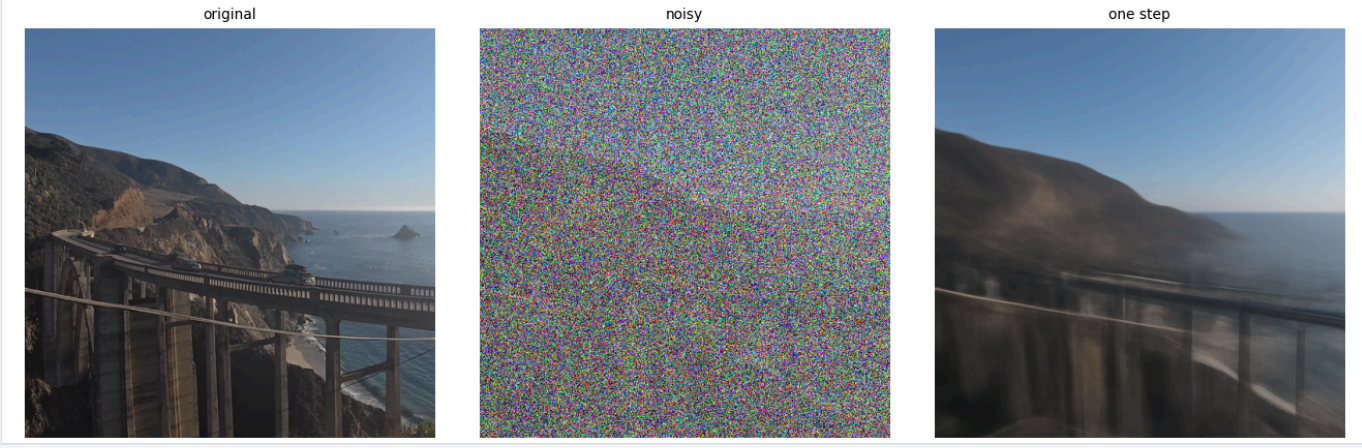
Blurring removes noise and image detail at the same time. At t = 0.75 the result is a soft but recognizable bridge. At t = 0.25 a strong blur is needed to suppress the noise, and almost all structure is lost with it.

A.1.3: One-step denoising

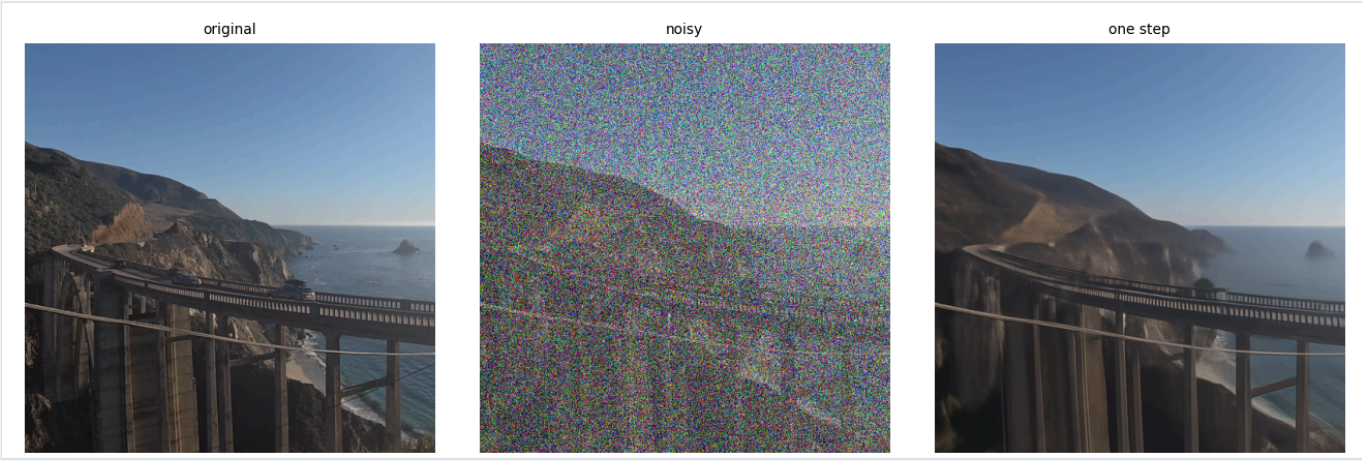
The flow model predicts the velocity v at the noisy image. Following that velocity for the remaining time 1 − t in one single step gives an estimate of the clean image:

$\hat{x}_1 = x_t + (1 - t)\, v_\theta(x_t, t, c)$

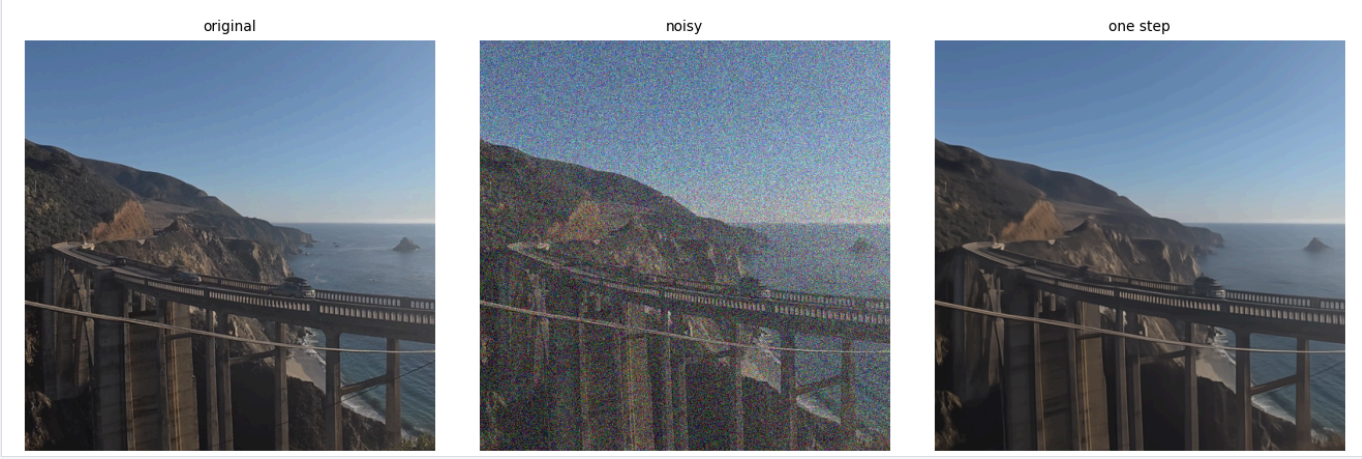
```
def one_step_denoise(x, t, condition, predict_velocity):
    return x + (1 - t) * predict_velocity(x, t, condition)
```



t = 0.25: original, noisy input, one-step estimate



t = 0.50



t = 0.75

The one-step estimate is much better than the Gaussian blur, especially at high noise levels. At t = 0.50 its mean squared error to the original is 0.0042, compared to 0.0119 for the best blur. At t =

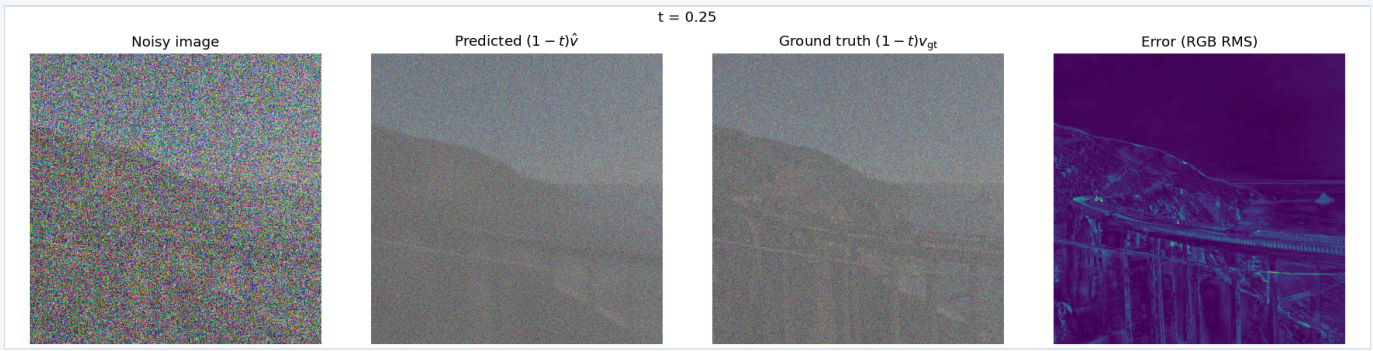
0.75 it is close to the original. At t = 0.25 it is blurry: with so little information left, the best single guess is an average of all images that could have produced this noisy input.

A.1.4: Visualizing velocity

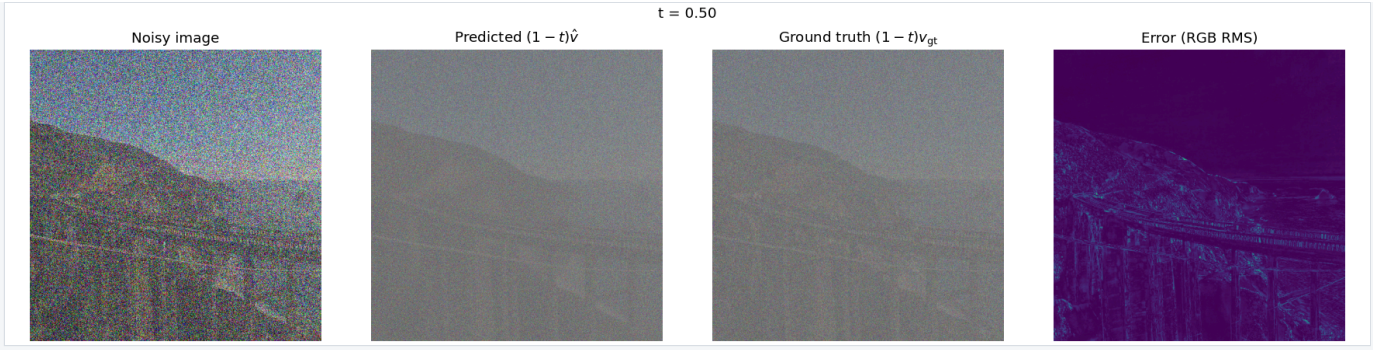
Because the original image and the noise are known, the exact update is also known: `(1 – t) v_gt = x₁ – x_t`. The figures compare it with the model's predicted update `(1 – t) v̂`. Both updates are shown on the same symmetric color scale with zero at middle gray. The error map is the RMS difference of the two updates over the RGB channels at each pixel, with the same color limits for all three times.

```
predicted_update = (1 - t) * predict_velocity(x_t, t, condition)
ground_truth_update = clean_image - x_t
error_map = torch.sqrt(torch.mean((predicted_update - ground_truth_update) ** 2, dim=1))
```

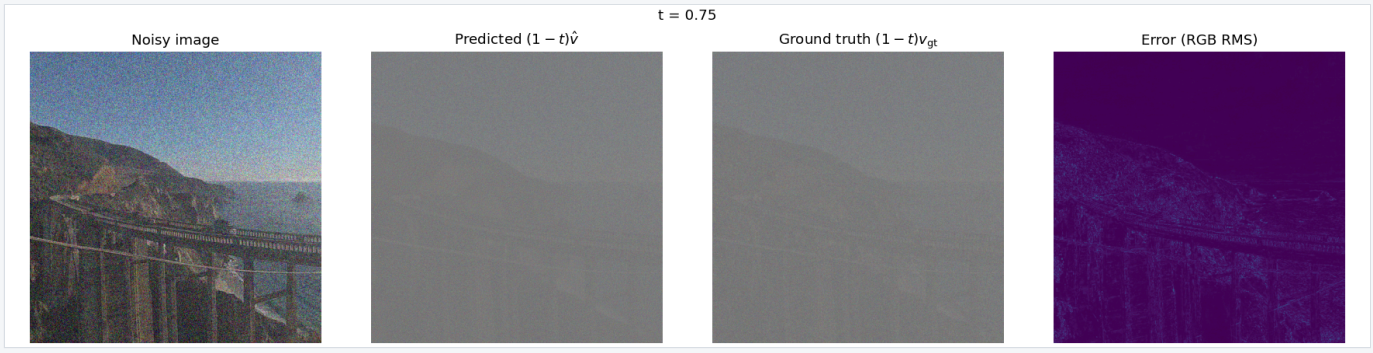
original image



t = 0.25: noisy image, predicted update, ground-truth update, error



t = 0.50



t = 0.75

The error is largest at t = 0.25 and shrinks as t grows. The model can remove the noise, but it cannot know which fine details the original image had, so the error concentrates on edges and textures such as the bridge's railing and the rocks.

A.1.5: Iterative denoising with Euler sampling

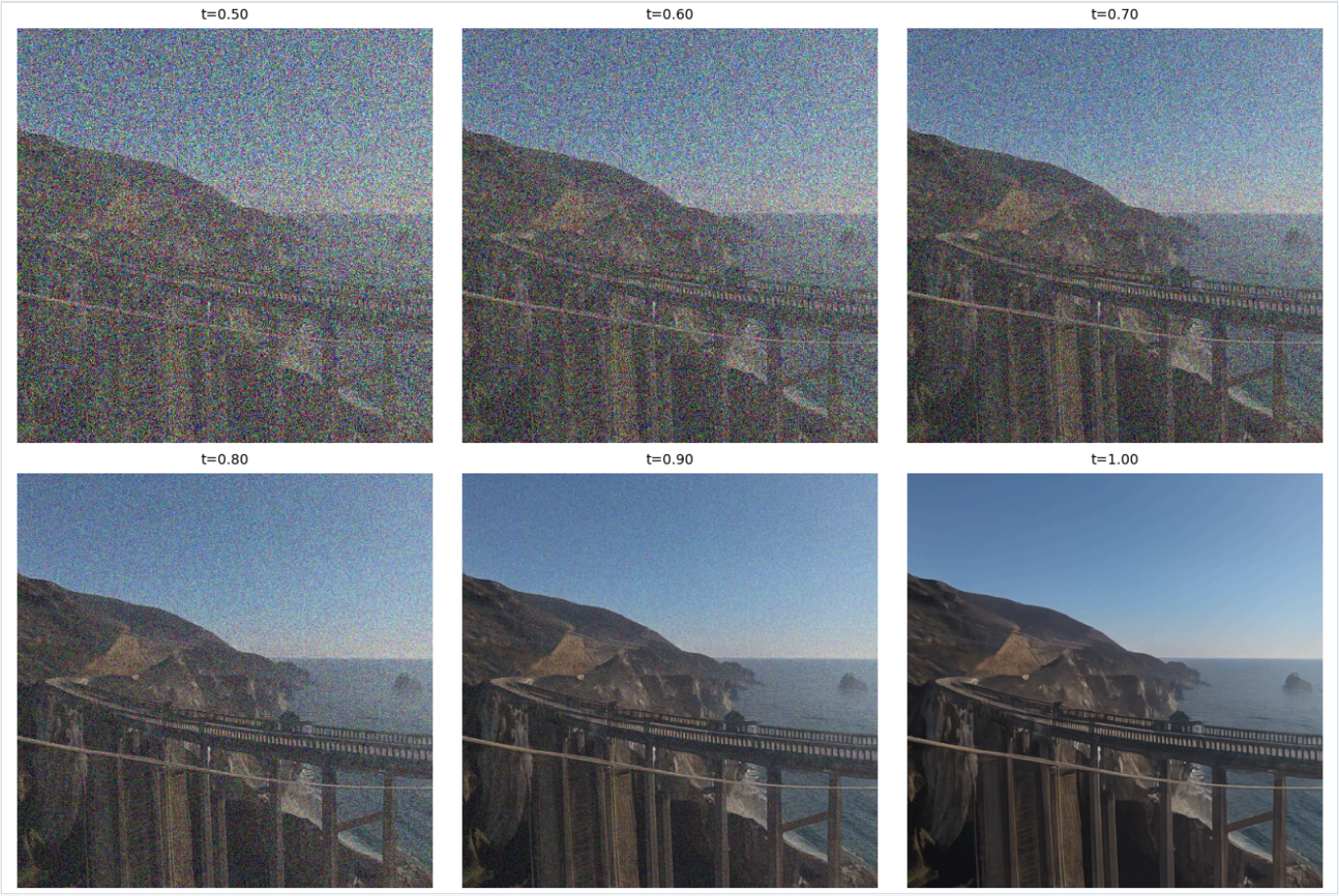
Instead of one big step, Euler integration takes many small steps of size h = 0.02 and predicts a new velocity after every step:

`x_{t+h} = x_t + h v_θ(x_t, t, c)`

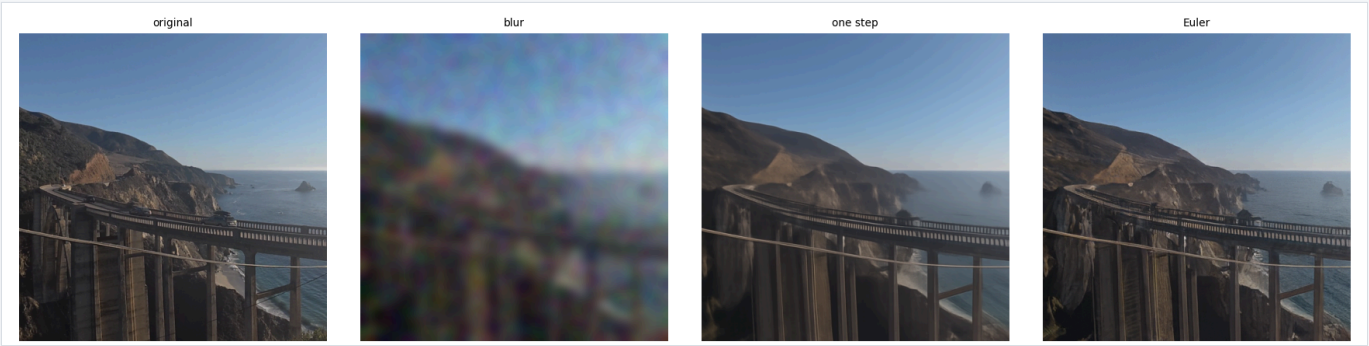
The last step is shortened if needed so that the sampler ends exactly at t = 1. Starting at t = 0.5, this takes 25 steps.

```
def euler_sample(x_start, t_start, step_size, velocity):
    x, t = x_start, t_start
    states = [(t, x)]
    count = 0
    while 1 - t > 1e-8:
        h = min(step_size, 1 - t)
        x = x + h * velocity(x, t)
        t += h
        count += 1
```

```
        if count == 5:
            states.append((t, x))
            count = 0
    states.append((1.0, x))
    return x, states
```



Euler trajectory from t = 0.50, every fifth step

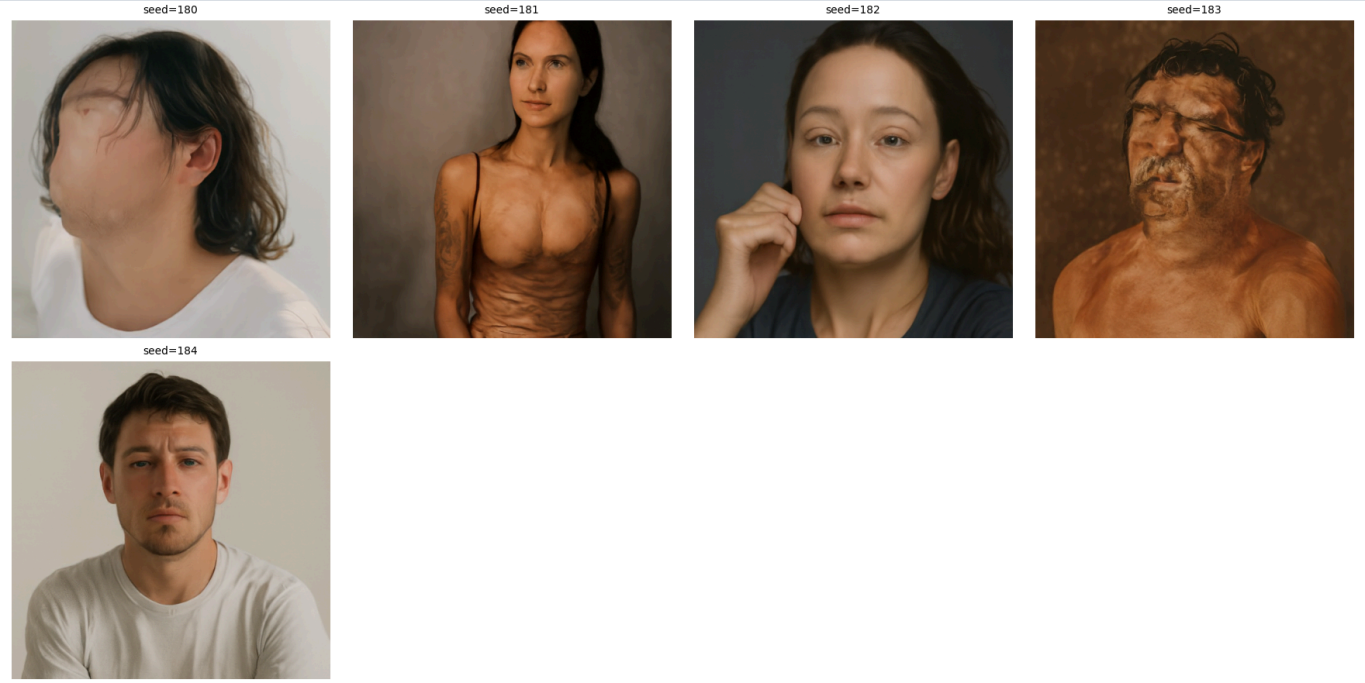


original, Gaussian blur, one-step estimate, Euler denoising (all from t = 0.50)

Euler denoising gives by far the sharpest result. It is not identical to the original: the model has to invent the details that the noise destroyed, so small structures look plausible but differ from the real photo. This is also why its mean squared error (0.0053) is slightly higher than that of the one-step estimate (0.0042): the one-step estimate is blurry, but a blurry average is closer to the original in MSE than sharp, invented details.

A.1.6: Generating images from noise

Starting the same sampler at t = 0 with pure Gaussian noise generates new images from scratch. All five samples use the prompt `"a high quality photo"` and the seeds 180 to 184.



five samples without guidance

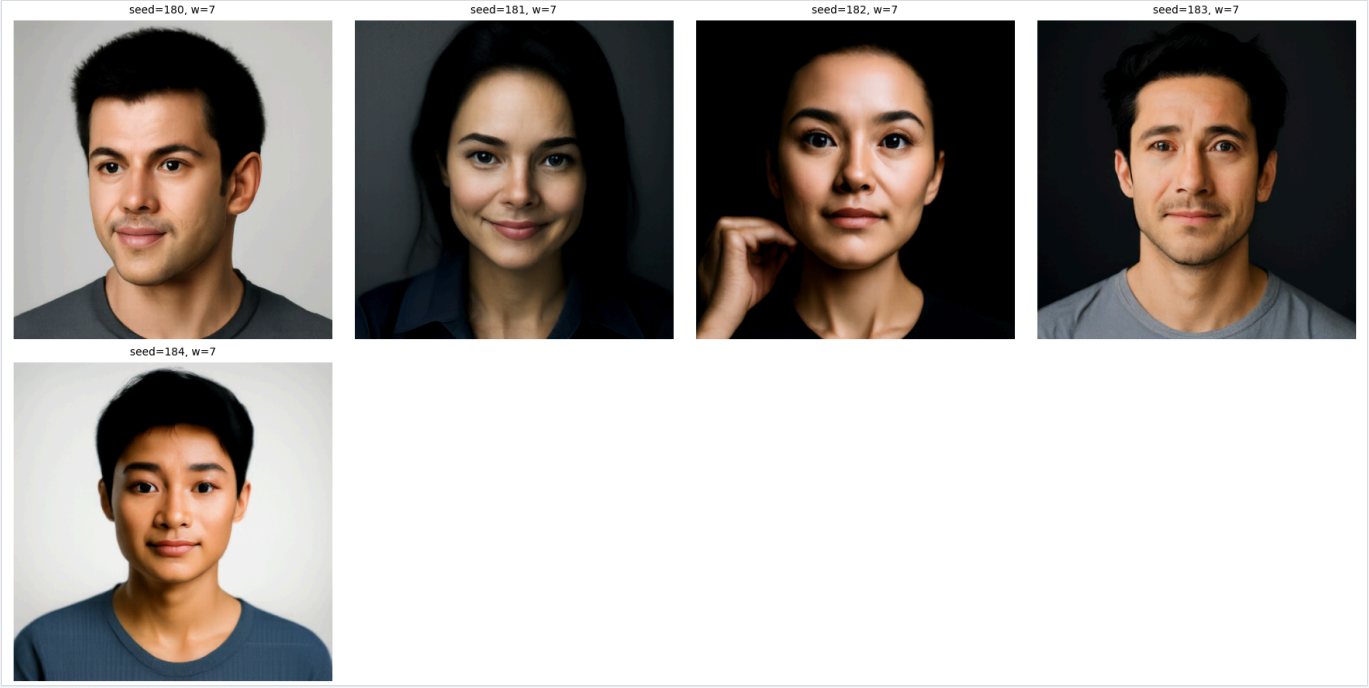
All five samples are portraits of people, so the generic prompt clearly favors people photos. The skin, hair and lighting look photographic, but the images are often not coherent: in seed 180 the face is missing and blends into the hair, and in seed 183 the face and skin look distorted. Seeds 182 and 184 are fairly realistic portraits.

A.2.1: Classifier-free guidance

CFG evaluates the model twice per step, once with the prompt (v_c) and once with the empty prompt (v_u), and pushes the result further in the direction of the prompt:

`v_cfg = v_c + w (v_c − v_u)` , with w = 7

```
def cfg_velocity(x, t, condition, w, predict_velocity, null_condition):
    v_c = predict_velocity(x, t, condition)
    v_u = predict_velocity(x, t, null_condition)
    return v_c + w * (v_c - v_u)
```



five CFG samples with the same seeds as in A.1.6

With guidance, every sample is a clean, well-formed portrait with sharp eyes, correct anatomy and a studio background, including the seeds that failed without guidance. The price is diversity: all five images are now centered head-and-shoulders portraits looking into the camera, with very similar lighting and composition, while the unguided samples varied more in pose and framing.

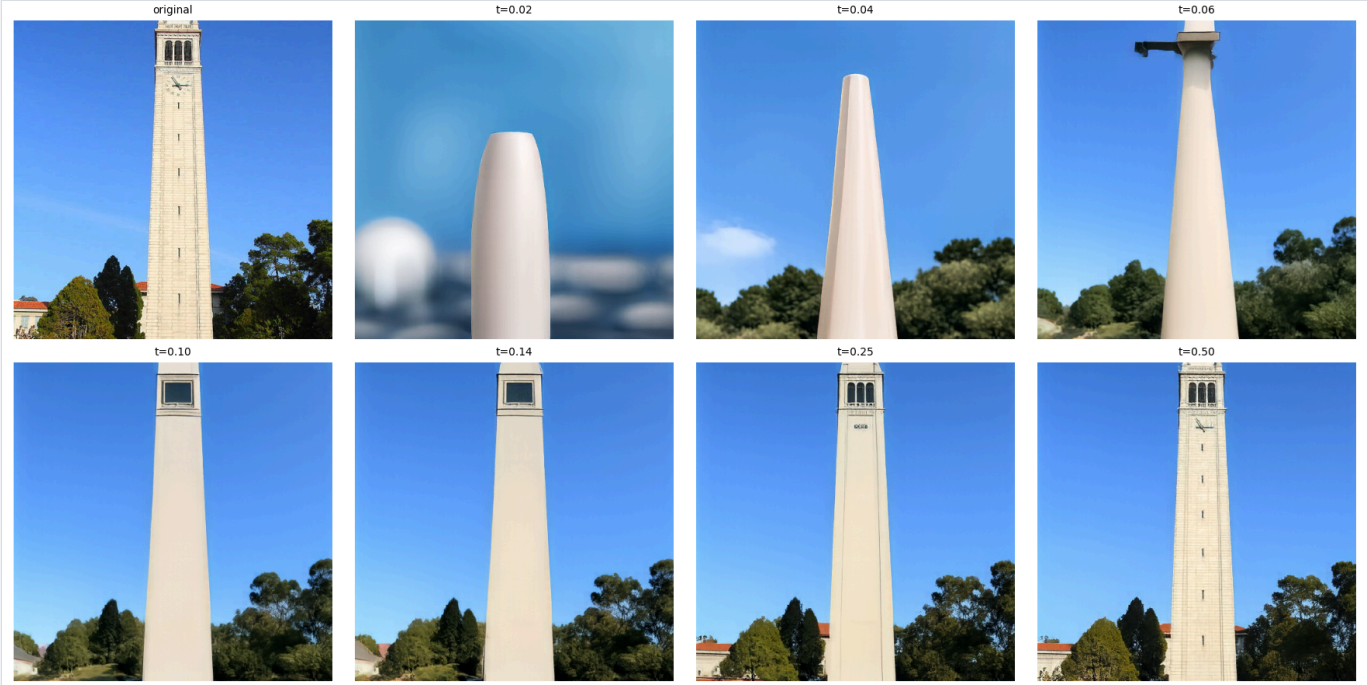
A.3.1: Image-to-image editing

To edit an image, it is first noised to a starting time t with the interpolation from A.1.1 and then denoised with Euler and CFG back to t = 1 (SDEdit). A small starting time means a lot of noise and a large edit, a large starting time keeps more of the original.

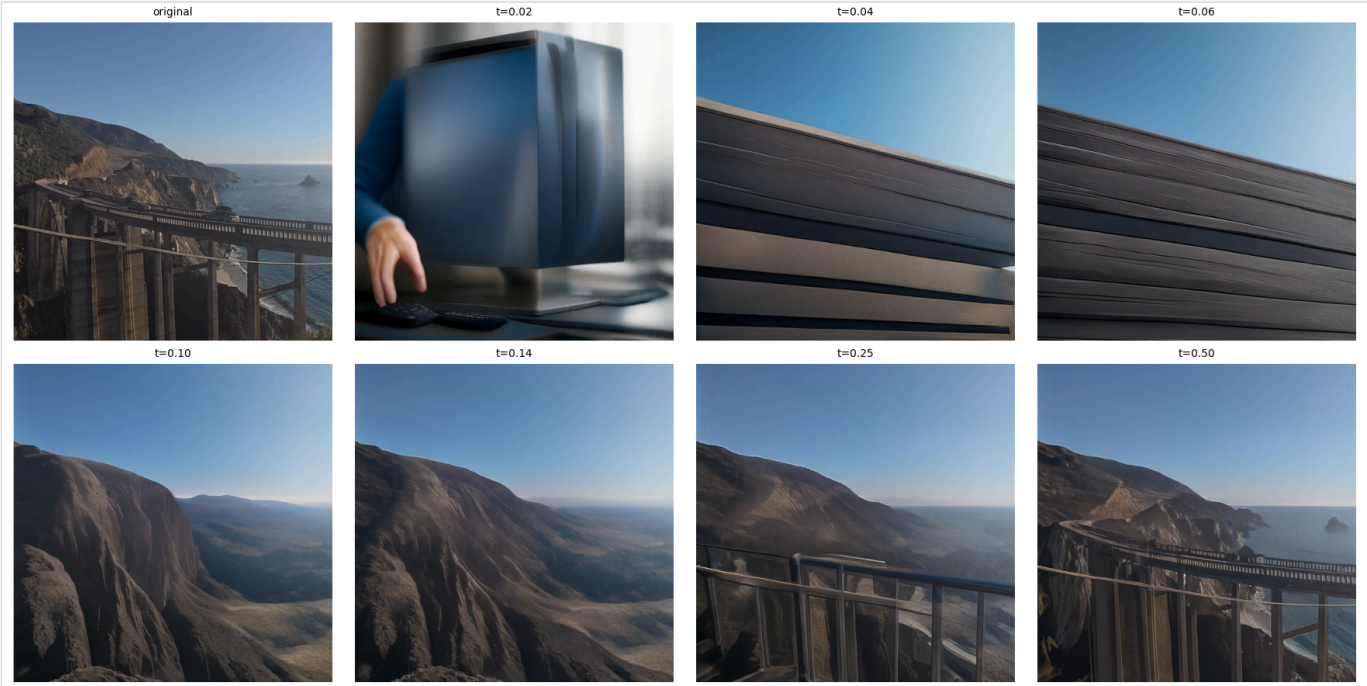
```
def edit(image, noise, t_start, condition, predict_velocity, w=7, step_size=0.02):
    x_start = interpolate(image, noise, t_start)
    x, _ = euler_sample(x_start, t_start, step_size,
                        lambda x, t: cfg_velocity(x, t, condition, w,
                                                  predict_velocity, null_condition))
    return x
```

Starting times 0.02, 0.04, 0.06, 0.10, 0.14, 0.25, 0.50, prompt "a high quality photo".

Campanile



Bixby Creek Bridge



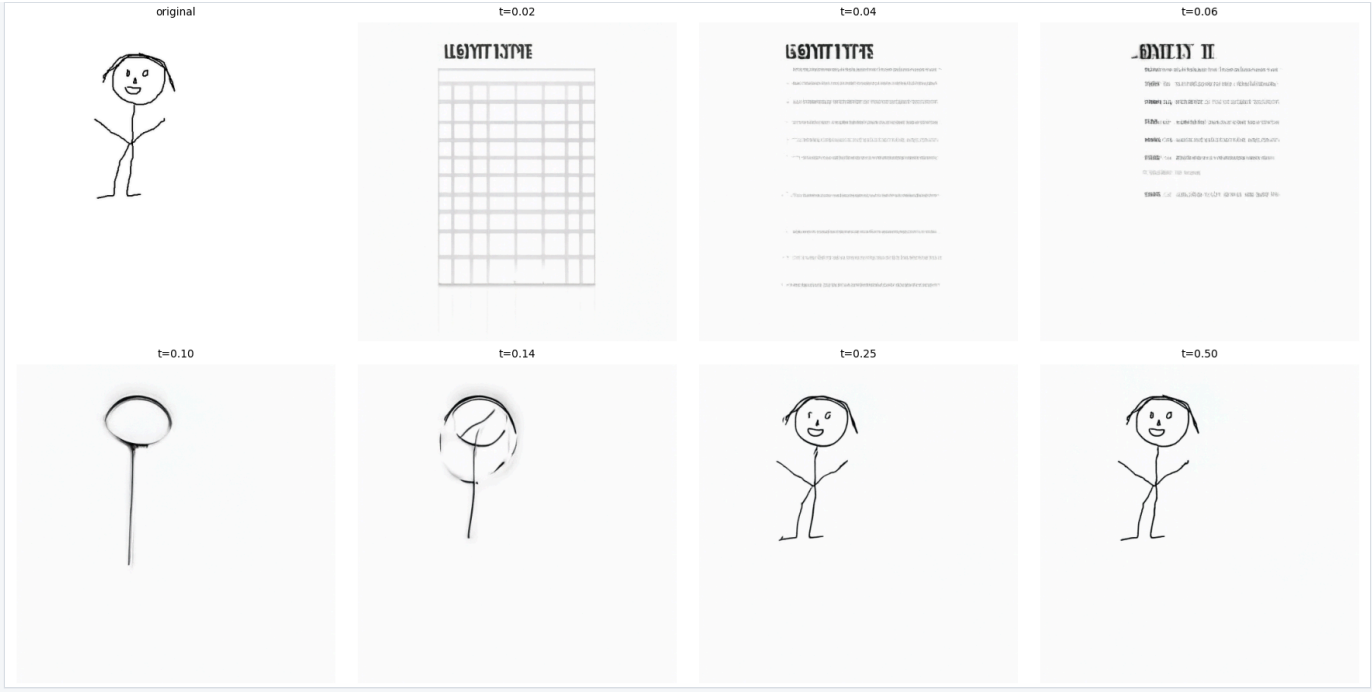
Yosemite

At t = 0.02 the sampler generates an almost unrelated image. For the bridge photo it produced a computer monitor, and only the blue color of the sky survived. At t = 0.04 and 0.06 the bridge turns into a close-up of dark wooden planks against a blue sky, which keeps the diagonal shape of the bridge deck. At t = 0.10 and 0.14 the image becomes a coastal mountain landscape with the same horizon, but without the bridge. At t = 0.25 a railing appears, and only at t = 0.50 is the Bixby Bridge back, almost identical to the original.

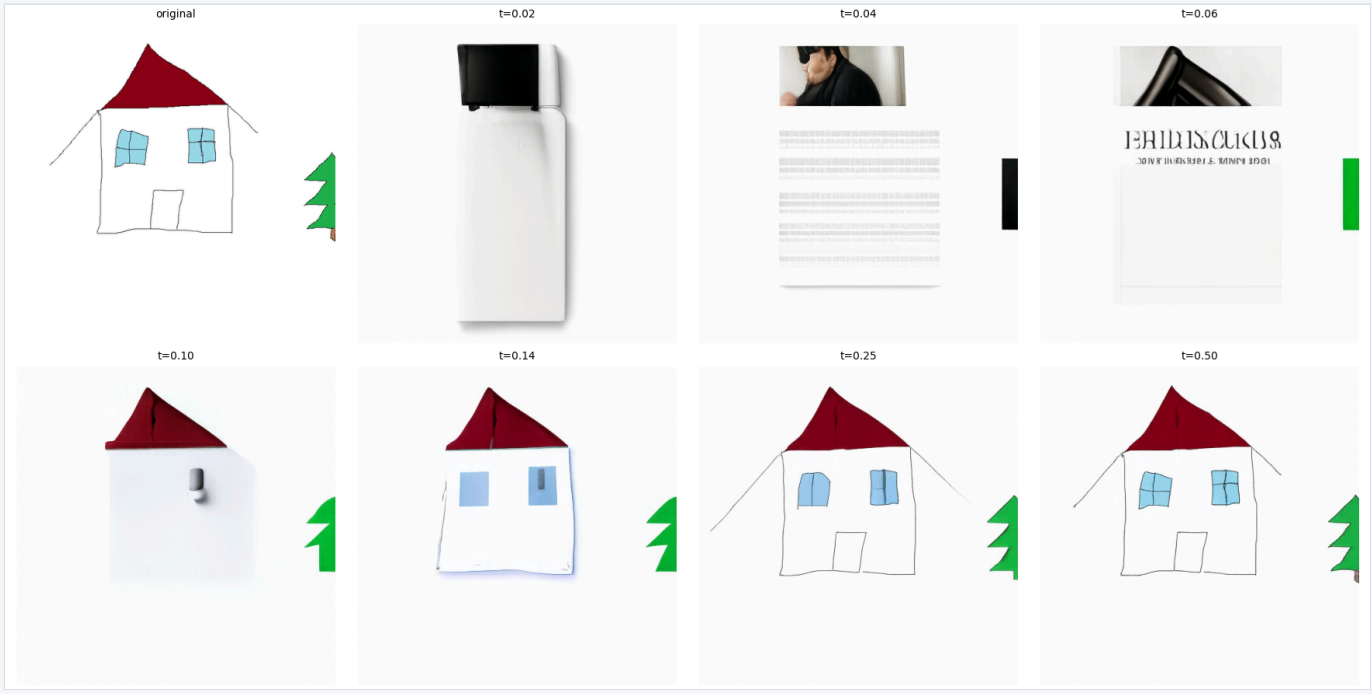
Web image and hand-drawn images

The same procedure projects non-realistic images onto the manifold of natural images.

image from the web



drawing 1



drawing 2

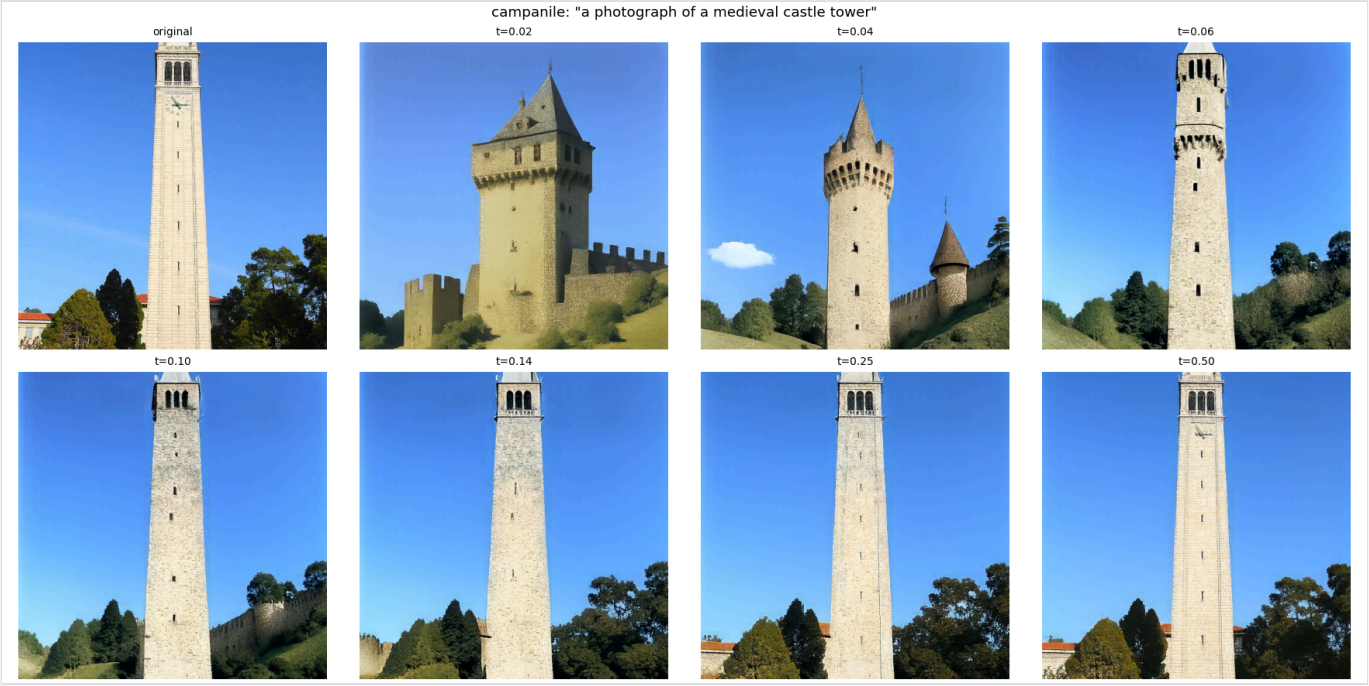
The web image is a photo of a castle on a hill at dusk. At t = 0.02 to 0.06 the model turns the dark blue and orange colors into a hand holding a camera or a red box. At t = 0.10 it becomes a modern building with lights reflected in the water, at t = 0.14 a lit rock formation, and from t = 0.25 on the hill with the castle walls is back.

The two drawings are mostly white, so at low starting times the model has almost nothing to hold on to and produces documents and printed text on a white page. For the stick figure, t = 0.10 and 0.14 keep only the round head and the vertical line and turn them into a lollipop- or racket-like object. For the house, t = 0.10 and 0.14 turn the drawing into a real object: a paper or cardboard house with a fabric-like red roof and blue windows, photographed on a white background. From t =

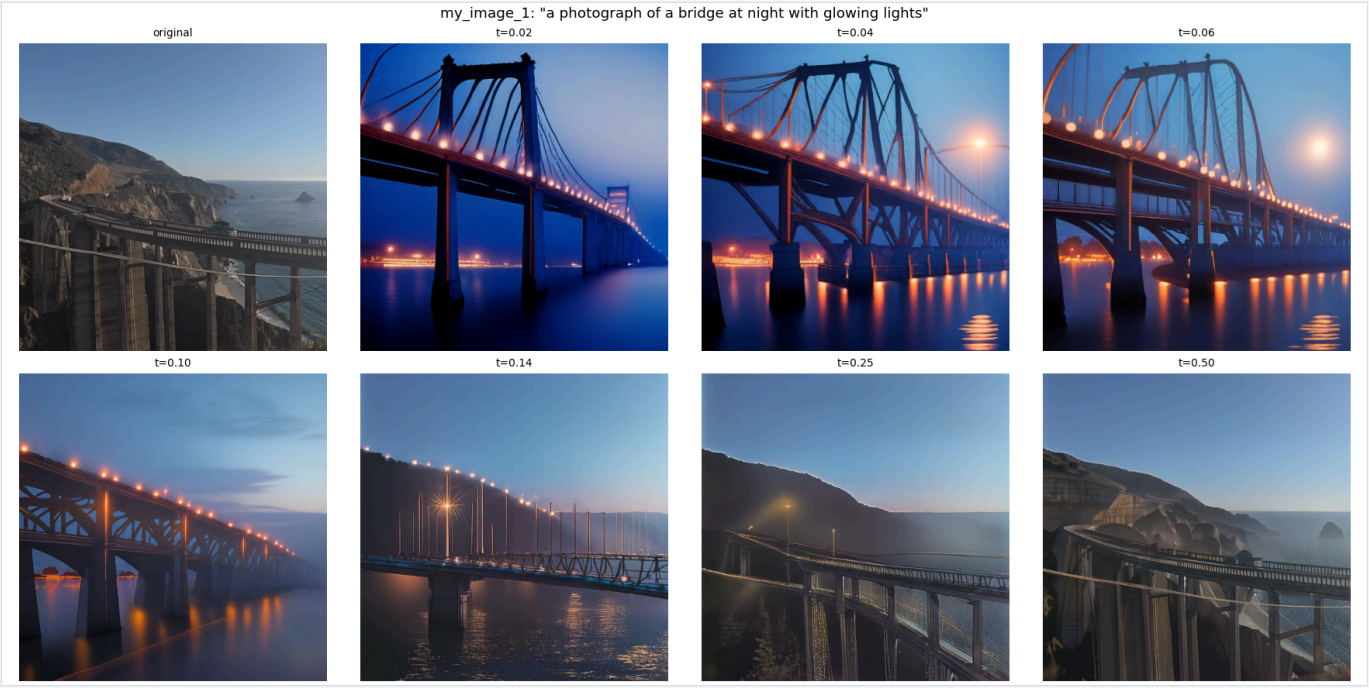
0.25 on, both drawings come back almost unchanged, because the model accepts simple line drawings on white paper as realistic images.

A.3.2: Text-guided image editing

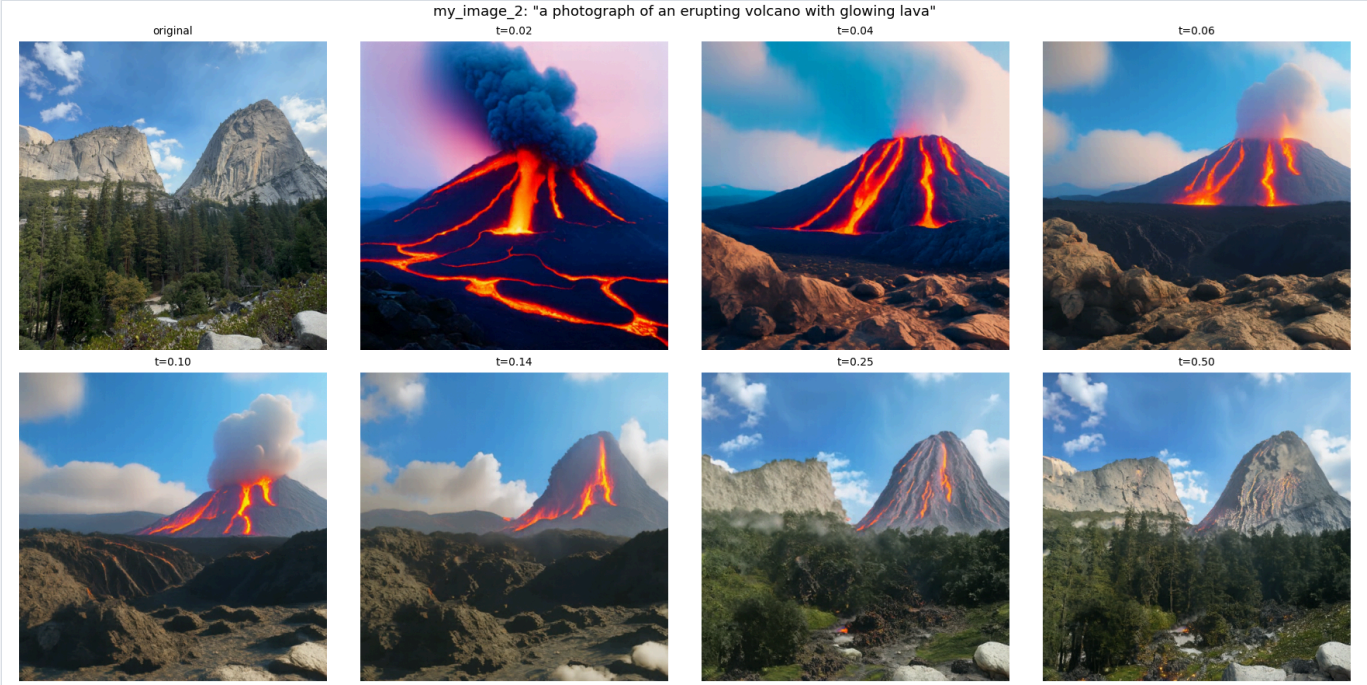
Replacing the generic prompt with a description of the desired edit makes the model change the image in that direction.



Campanile, "a photograph of a medieval castle tower"



Bixby Creek Bridge, "a photograph of a bridge at night with glowing lights"



Yosemite, "a photograph of an erupting volcano with glowing lava"

At small starting times the prompt wins and the result has little to do with the input. At large starting times the original wins and the prompt has almost no effect. The interesting edits are in between, where the layout of the original is kept but its content follows the prompt. The Yosemite volcano shows this best: at t = 0.02 it is a generic erupting volcano, at t = 0.10 and 0.14 the right-hand granite dome itself becomes the volcano with lava streams, and at t = 0.25 the whole valley with the forest and the left cliff is back while the dome still glows with lava.

A.3.3: Visual anagrams

A visual anagram looks like one prompt upright and like another one when rotated by 180°. In every Euler step, the CFG velocity for prompt a is computed on the image, and the CFG velocity for prompt b on the rotated image, which is then rotated back. The image moves in the direction of the average of both.

```
def visual_anagrams(noise, condition_a, condition_b, predict_velocity, w=7, step_size=0.02):
    def velocity(x, t):
        v_1 = cfg_velocity(x, t, condition_a, w, predict_velocity, null_condition)
        v_2 = rotate180(cfg_velocity(rotate180(x), t, condition_b, w,
                                     predict_velocity, null_condition))
        return (v_1 + v_2) / 2
    x, _ = euler_sample(noise, 0, step_size, velocity)
    return x
```

"an oil painting of a sailing ship" / rotated: "an oil painting of a flying bird"

"a watercolor of a rabbit" / rotated: "a watercolor of a duck"

The ship and bird anagram works in both orientations. Upright, the hull and sails of a ship sit in a stormy sea, and rotated, the sails become the wings of a seagull flying through the sky. The light sky and the dark sea simply swap roles. The rabbit and duck anagram works much better upright: the rabbit is clear, but rotated, the ears only weakly suggest a duck's beak and the result mostly looks like an upside-down rabbit. A rabbit's head and ears do not have enough in common with a duck's head and bill for both readings to be equally strong.

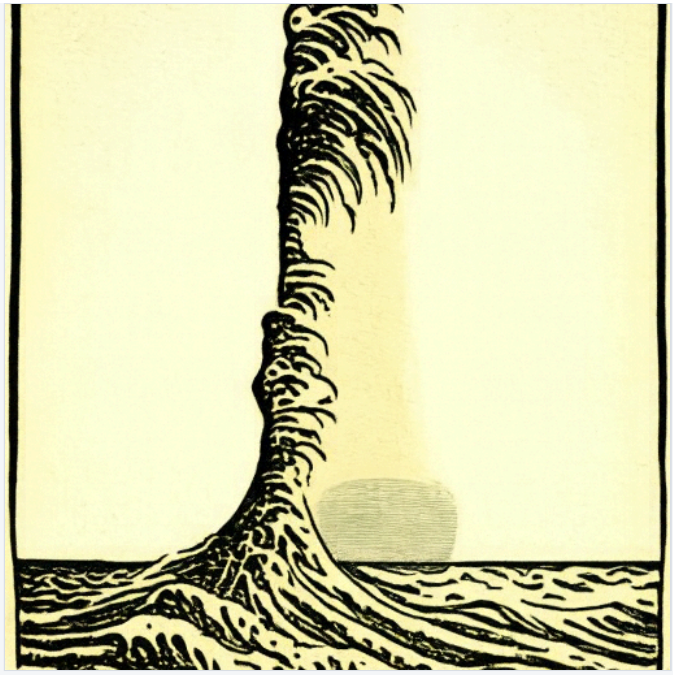
A.3.4: Hybrid images

As in Project 2, a hybrid image combines the low frequencies of one image with the high frequencies of another. Here the combination happens on the velocities in every Euler step: the low-pass of the velocity for the distant prompt plus the high-pass of the velocity for the close prompt. The low-pass is a Gaussian blur with kernel size 265 and σ = 16, the high-pass is the velocity minus its low-pass.

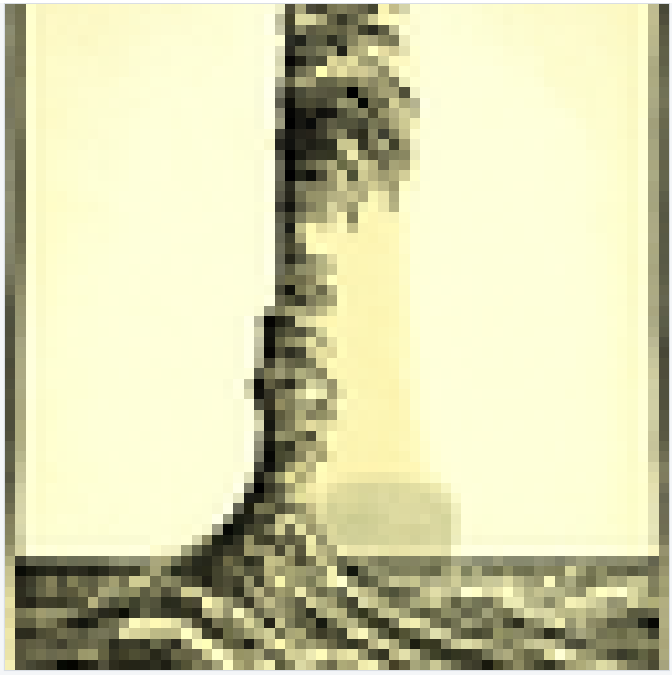
```
def make_hybrids(noise, condition_a, condition_b, predict_velocity, w=7, step_size=0.02):
    def velocity(x, t):
        v_1 = cfg_velocity(x, t, condition_a, w, predict_velocity, null_condition)
        v_2 = cfg_velocity(x, t, condition_b, w, predict_velocity, null_condition)
        return lowpass(v_1) + (v_2 - lowpass(v_2))
    x, _ = euler_sample(noise, 0, step_size, velocity)
    return x
```

close: "a lithograph of a jungle"

downsampled to 64×64 (far away): "a lithograph of a tiger"



close: "a lithograph of ocean waves"



downsampled to 64×64 (far away): "a lithograph of a lighthouse"

Up close, both images clearly show the close prompts: a jungle with palm trees and leaves, and a tall breaking wave. From far away the illusions are weaker. The tall vertical shape of the wave works as the outline of a lighthouse, and the round glow next to it looks like its light, so the second hybrid works reasonably well. The tiger is hard to see in the first one: the low frequencies mostly show a light background with dark vertical bands, which match a tiger's stripes but not its shape.

Part B: Flow matching from scratch

B.1.1: Implementing the UNet

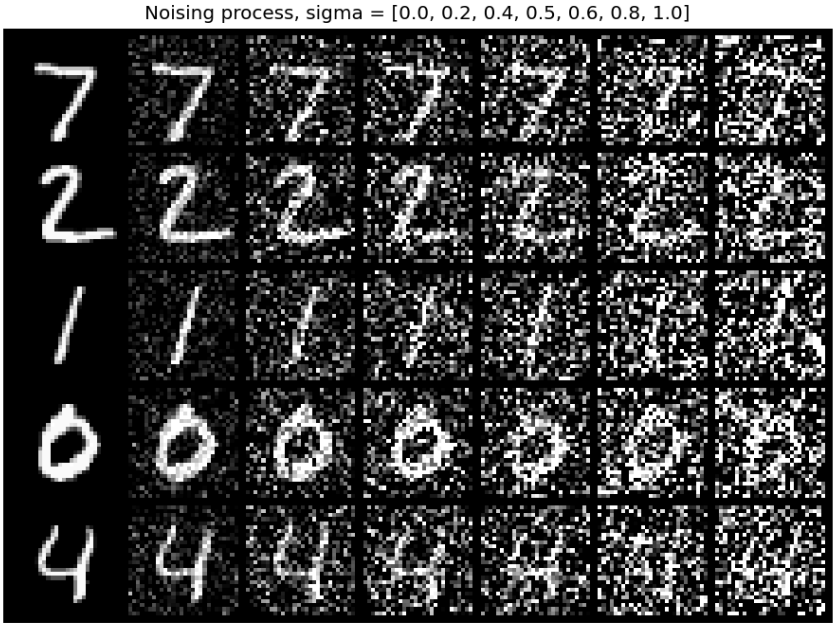
The denoiser is a small UNet for 28×28 MNIST digits. The encoder halves the resolution twice (28 → 14 → 7) and averages the 7×7 feature map down to 1×1. The decoder upsamples back to 28×28. At every resolution, the decoder concatenates its features with the encoder features of the same size (skip connections), so fine detail that was lost in the bottleneck can be recovered. All blocks are built from Conv2d / ConvTranspose2d, BatchNorm and GELU.

```
x0 = self.conv_in(x)                          # D × 28 × 28
x1 = self.down1(x0)                           # D × 14 × 14
x2 = self.down2(x1)                           # 2D × 7 × 7
x3 = self.flatten(x2)                         # 2D × 1 × 1
x4 = self.unflatten(x3)                       # 2D × 7 × 7
x5 = self.up1(torch.cat([x4, x2], dim=1))     # D × 14 × 14
x6 = self.up2(torch.cat([x5, x1], dim=1))     # D × 28 × 28
```

```
x7 = self.conv_out(torch.cat([x6, x0], dim=1))    # D × 28 × 28
return self.final(x7)                             # 1 × 28 × 28
```

B.1.2: Training a denoiser

Training pairs are made by adding Gaussian noise to clean digits: `z = x + σ ε`. The figure shows this for σ from 0 to 1.

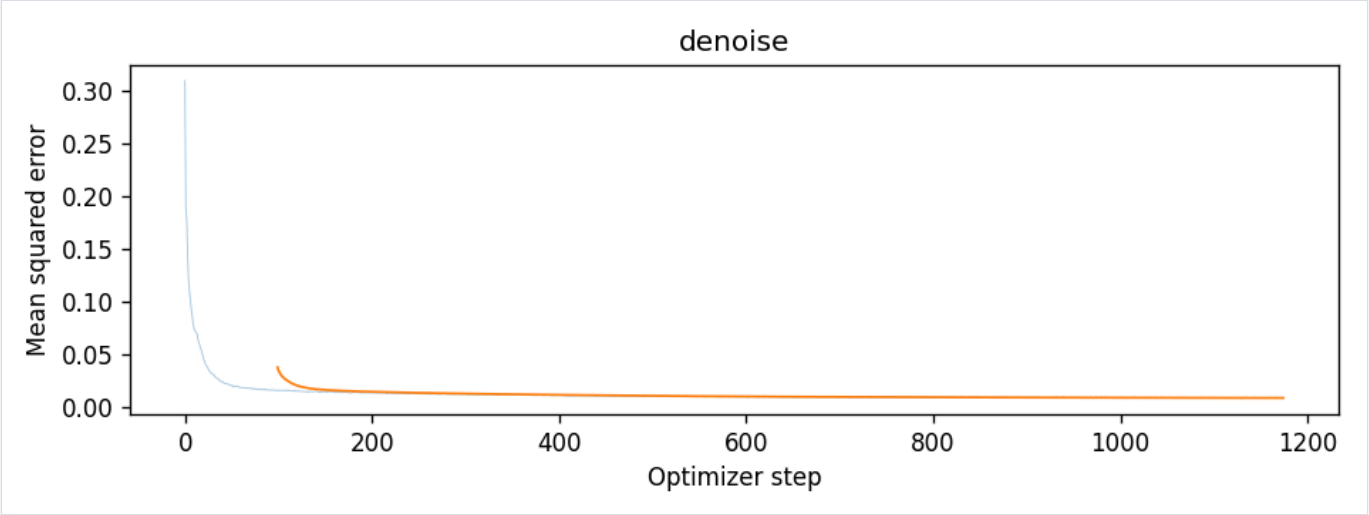


noising process, σ = 0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0

Training with σ = 0.5

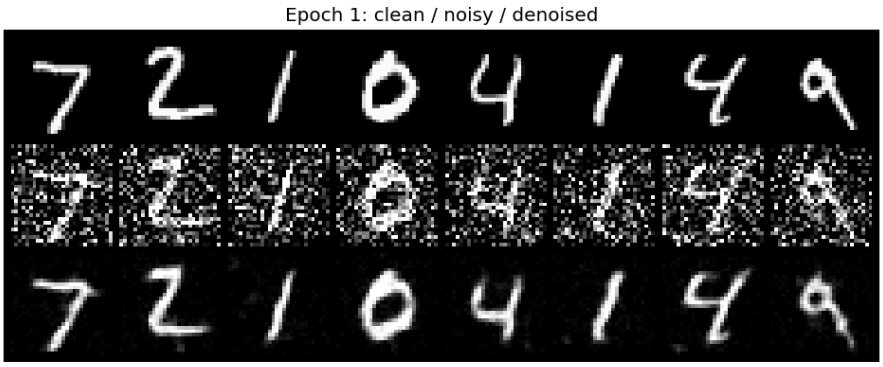
SETTING	VALUE
Hidden dimension D	128
Batch size	256
Optimizer	Adam, lr = 1e-4
Epochs	5
Loss	MSE between output and clean digit

The noise is sampled anew for every batch, so the network sees different noisy versions of the same digits in every epoch.

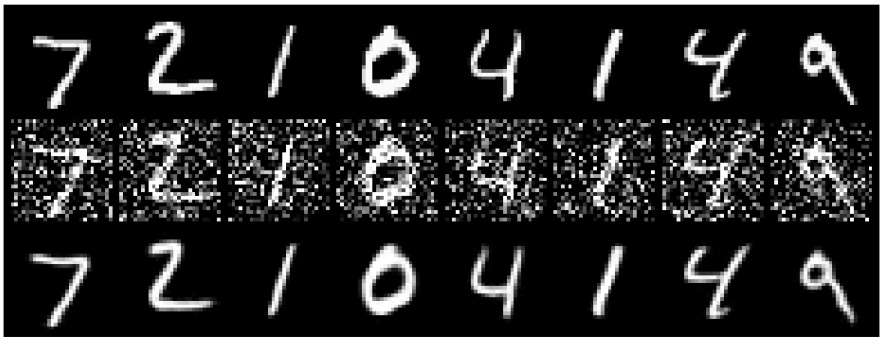


training loss

The loss falls steeply from 0.08 to about 0.015 during the first 100 steps and then slowly decreases to about 0.008 by the end of epoch 5.



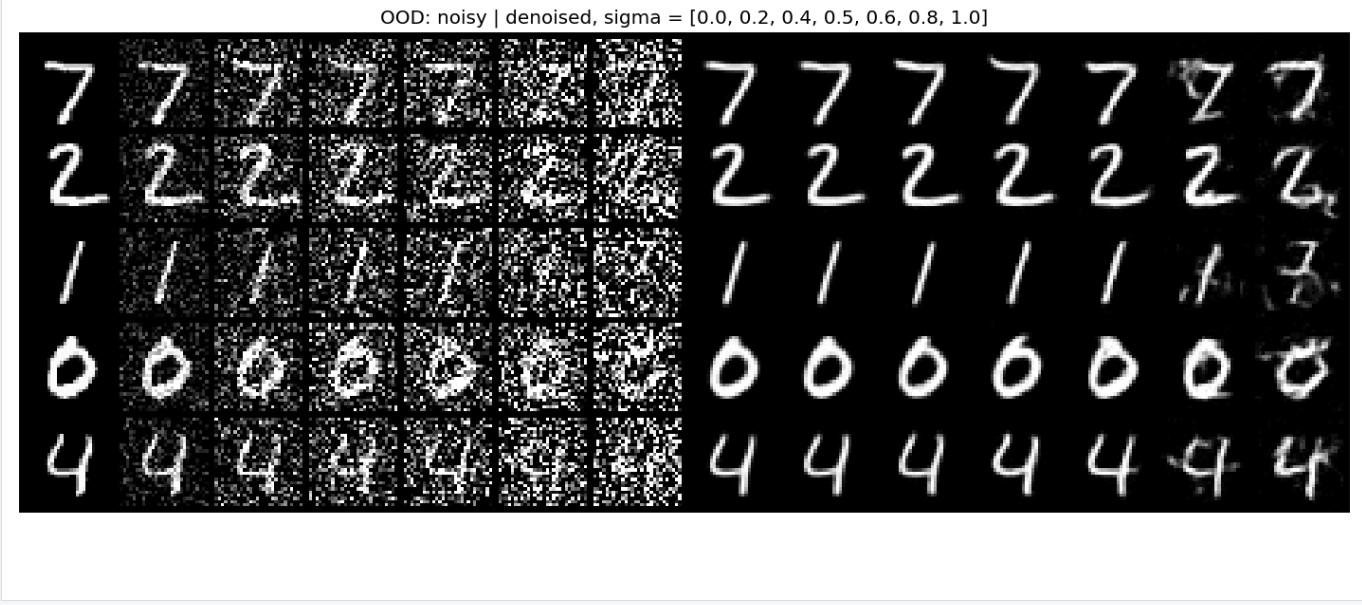
test set after epoch 1 (separate one-epoch run with the same settings): clean / noisy / denoised



test set after epoch 5: clean / noisy / denoised

After one epoch the digits are already recognizable, but the outputs are a bit blurry and some faint speckles of the noise remain in the background. After five epochs the background is clean and the strokes are smooth and close to the originals.

Out-of-distribution testing

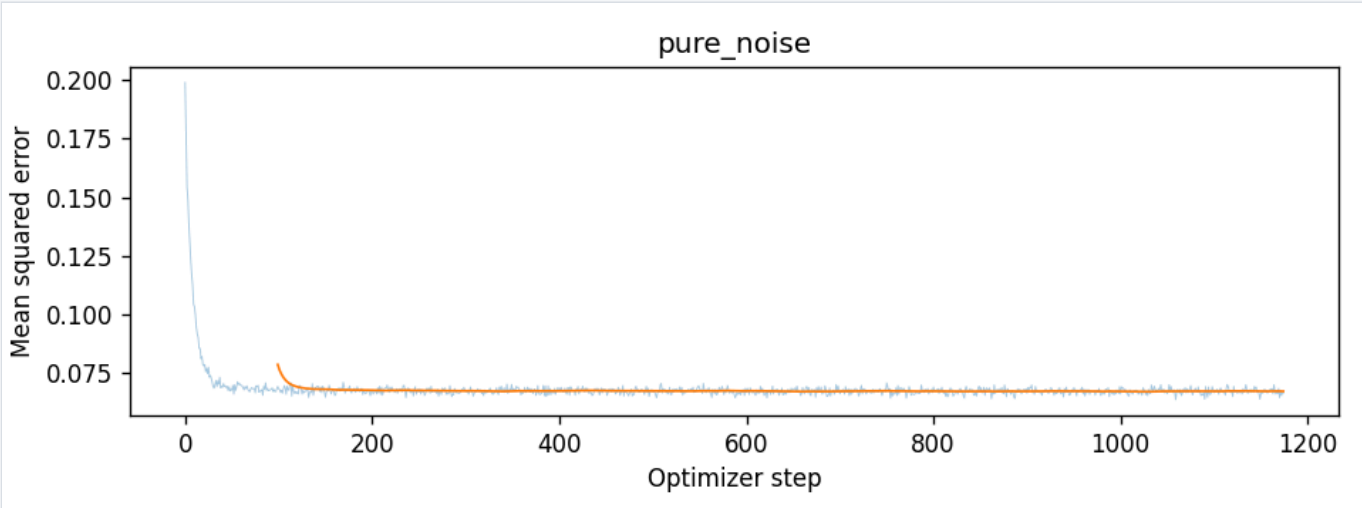


left: noisy inputs, right: denoised, σ = 0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0

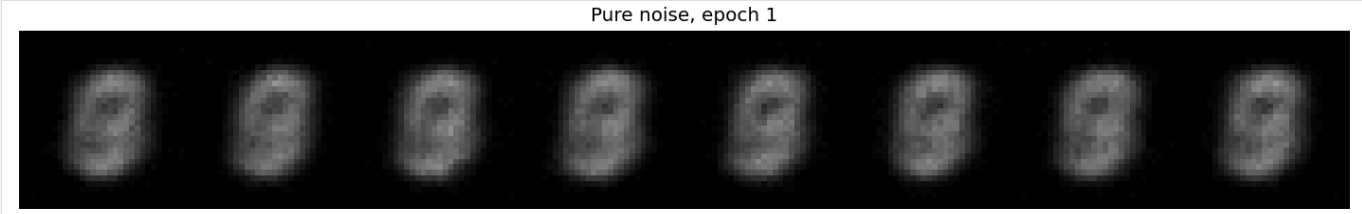
Up to σ = 0.6 all digits are denoised cleanly, so the model generalizes to noise levels slightly above the one it was trained on. At σ = 0.8 the first errors appear: the 1 breaks apart and the 4 starts to look like a 9. At σ = 1.0 most digits fail and the model invents strokes that are not there. Only thick, simple shapes such as the 0 and the 2 survive. The model has only learned to remove noise of strength 0.5, and the further the input is from that, the worse it gets.

Denoising pure noise

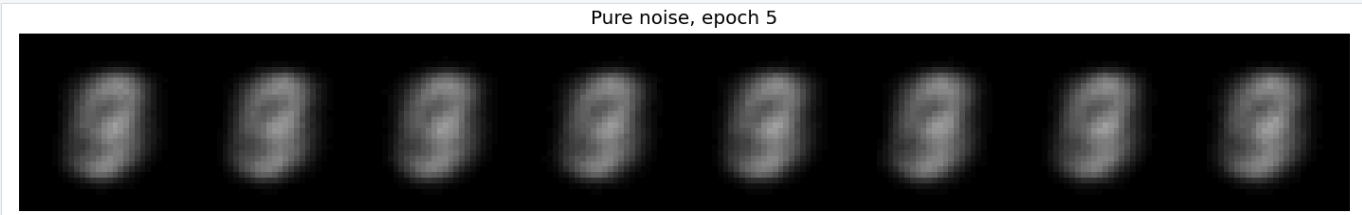
Training the same network with pure noise as input (no image at all) turns denoising into a generation task.



training loss



outputs after epoch 1



outputs after epoch 5

All outputs look the same: a blurry blob that looks like a mix of 0, 3, 8 and 9. Because the input contains no information about which digit to produce, the output that minimizes the MSE over the whole training set is the average of all training digits, the centroid of the data. The network learns to output this mean image regardless of the noise. This is why one-step denoising cannot generate new samples, and why iterative denoising is needed.

B.2.1: Adding time conditioning

For flow matching, the noisy input is again `x_t = (1 − t) x₀ + t x₁` with $x_0 \sim N(0, I)$, and the UNet learns to predict the velocity `x₁ − x₀`. The network needs to know t, so t is passed through two FCBlocks (Linear, GELU, Linear). Their outputs multiply the unflatten features (2D channels) and the first upsampling block (D channels), channel by channel.

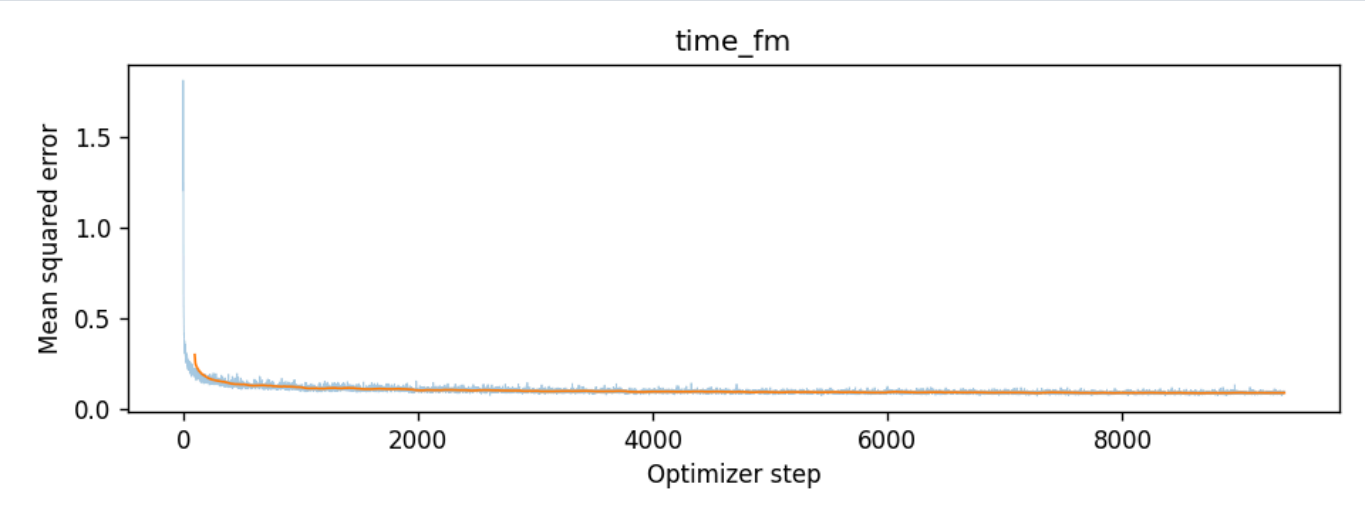
```
t = t.view(-1, 1)
t1 = self.fc1_t(t).view(-1, 2 * self.num_hiddens, 1, 1)
t2 = self.fc2_t(t).view(-1, self.num_hiddens, 1, 1)
...
x4 = self.unflatten(x3) * t1
x5 = self.up1(torch.cat([x4, x2], dim=1)) * t2
```

B.2.2: Training the time-conditioned UNet

Every training step samples a batch of digits, noise and a random `t ~ U[0, 1]` per image, builds x_t and minimizes the MSE between the prediction and `x₁ − x₀`.

```
x_0 = torch.randn_like(x_1)
t = torch.rand(x_1.shape[0], device=device)
t_img = t.view(-1, 1, 1, 1)
x_t = (1 - t_img) * x_0 + t_img * x_1
return F.mse_loss(unet(x_t, t), x_1 - x_0)
```

SETTING	VALUE
Hidden dimension D	64
Batch size	64
Optimizer	Adam, lr = 1e-2
Scheduler	ExponentialLR, γ = 0.1^(1/10), stepped once per epoch
Epochs	10



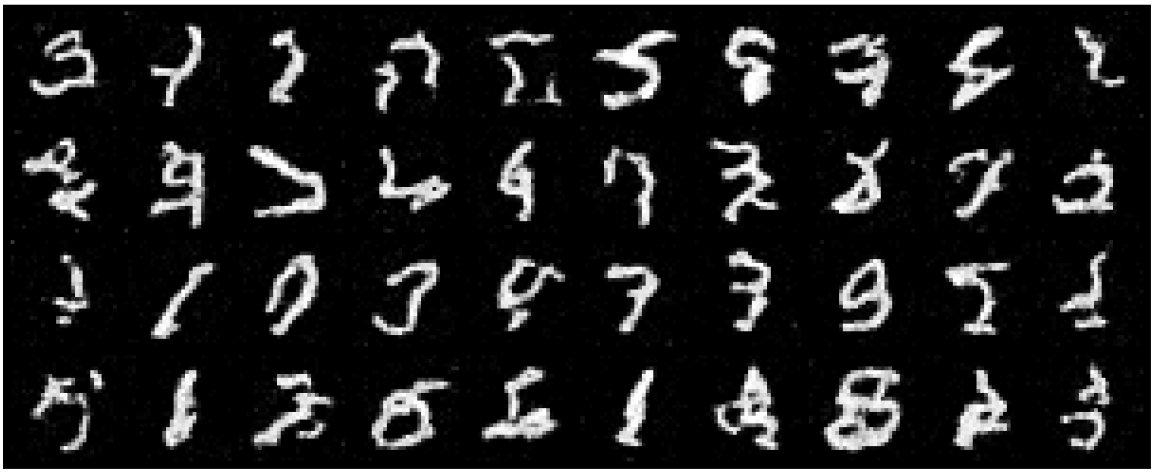
training loss, time-conditioned UNet

The loss starts at about 1.8 and drops below 0.2 within the first few hundred steps. After that it decreases only slowly and ends at about 0.09. It stays much higher than the denoiser's loss in Part 1, because the target $x_1 - x_0$ contains the random noise, which can never be predicted exactly.

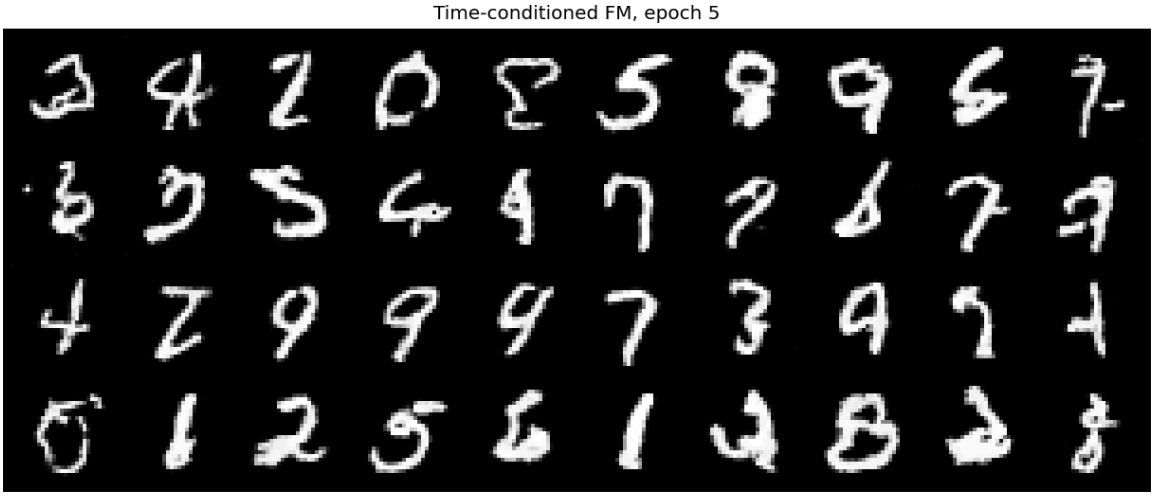
B.2.3: Sampling from the time-conditioned UNet

Sampling starts from pure noise at t = 0 and takes 50 Euler steps of size 1/50, the same as in Part A.

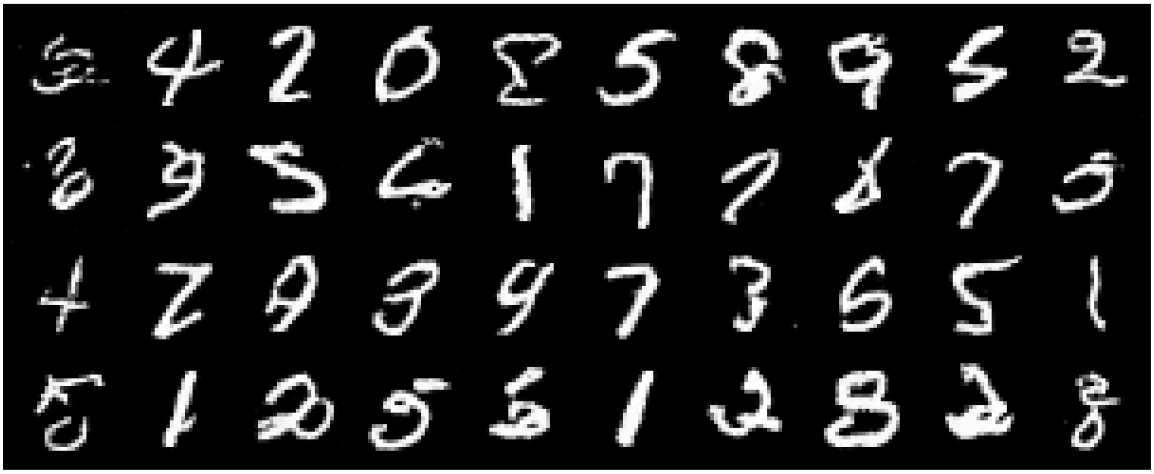
```
for i in range(num_ts):
    t = torch.full((N,), i * h, device=device)
    x = x + h * unet(x, t)
```



epoch 1

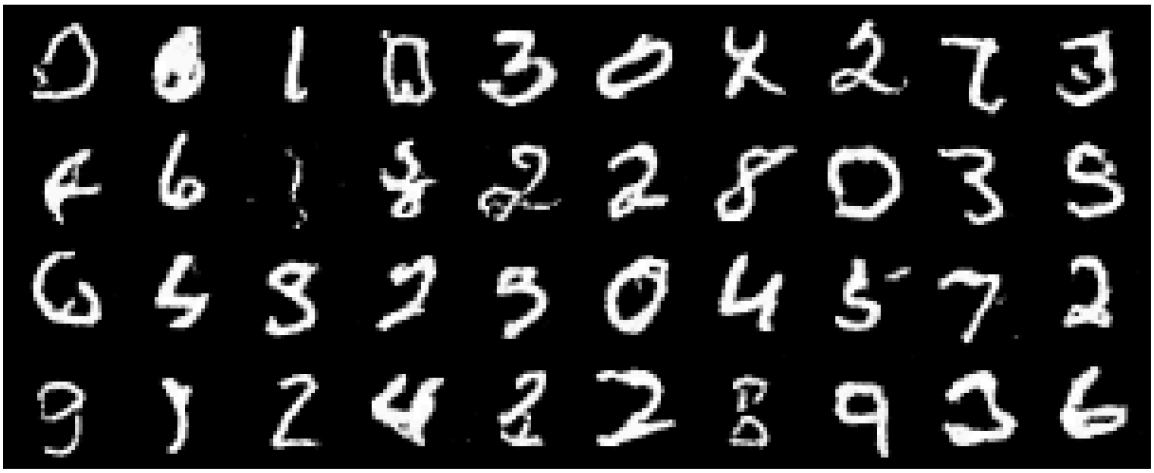


epoch 5



epoch 10

Time-conditioned FM, final model (seed 1)



final model, different seed

The same seed is used at every epoch, so each position starts from the same noise. After epoch 1 the samples are already stroke-like, but most of them are wobbly scribbles and only a few are legible digits. By epoch 5 many positions have turned into clear digits such as 2, 5, 7 and 9, and by epoch 10 most samples are legible. A few are still broken, for example a shape like a Σ in the first row or digits with extra loops. Since the model is not told which digit to draw, it sometimes mixes features of several digits in one sample.

B.2.4: Adding class conditioning

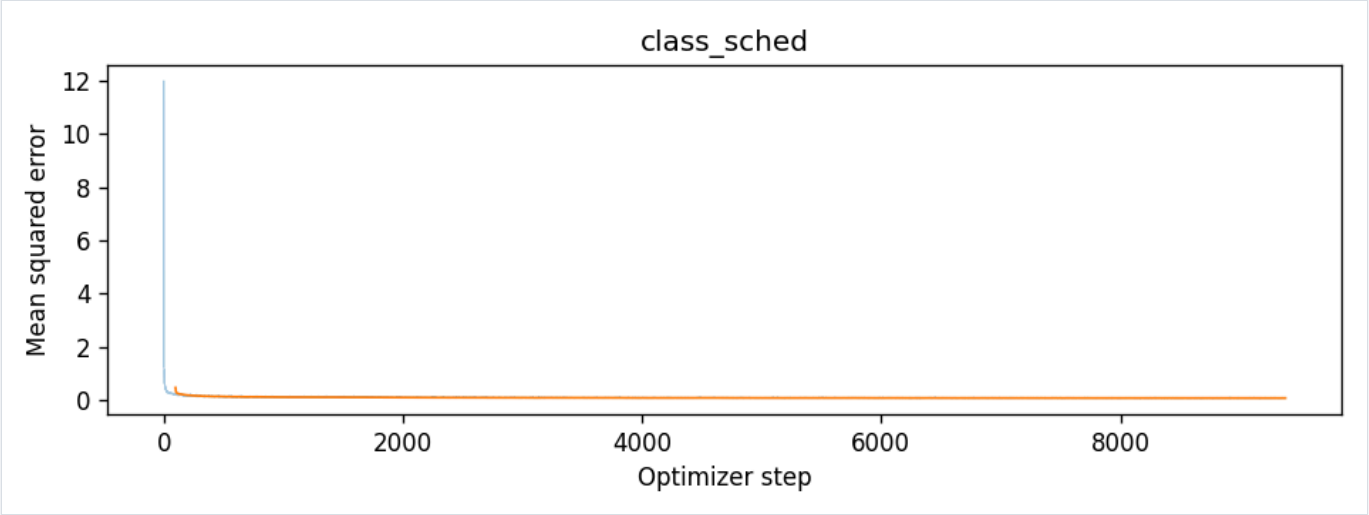
The digit class is given to the UNet as a one-hot vector through two more FCBlocks. The class embedding scales the features and the time embedding is added:

```
x4 = c1 * self.unflatten(x3) + t1
x5 = c2 * self.up1(torch.cat([x4, x2], dim=1)) + t2
```

During training, the class vector is set to zero for 10% of the batch (p_uncond = 0.1). This way the same network also learns the unconditional velocity, which is needed for classifier-free guidance.

B.2.5: Training the class-conditioned UNet

Training uses the same settings as the time-conditioned model (D = 64, batch size 64, Adam with lr = 1e-2 and exponential decay, 10 epochs).



training loss, class-conditioned UNet with scheduler

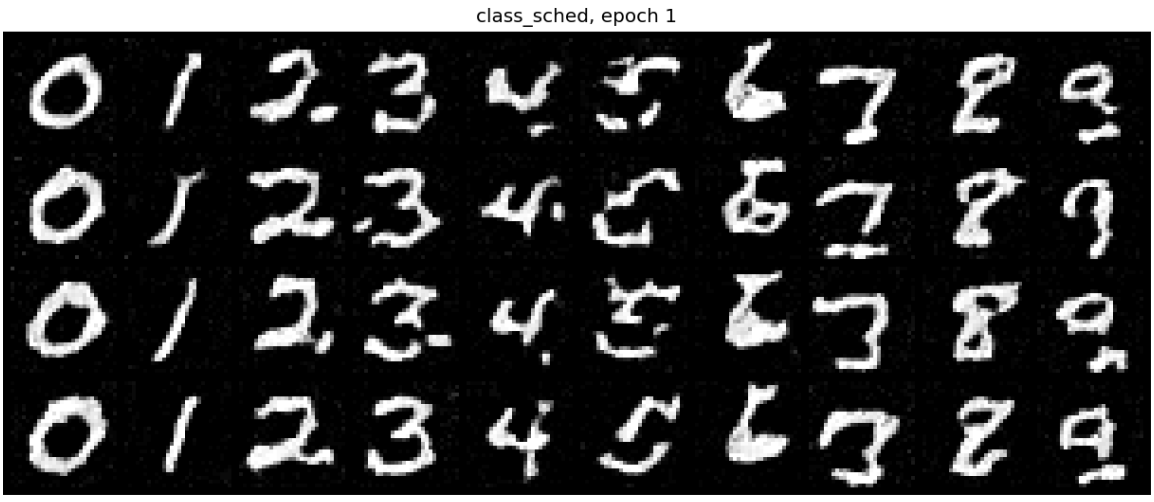
The very first steps have a loss of up to 12, because the initial learning rate of 1e-2 makes the first updates very large. The loss then falls to about 0.1 within a few hundred steps and stays there.

B.2.6: Sampling from the class-conditioned UNet

Sampling uses 300 Euler steps and classifier-free guidance with γ = 5. In Part B the guided velocity is written as `v = v_u + γ (v_c − v_u)`.

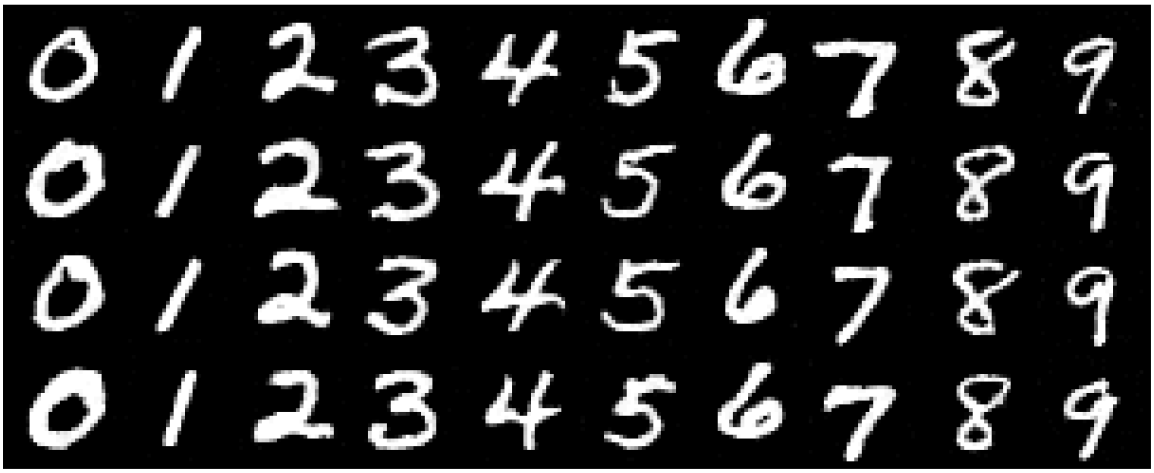
```
v_c = unet(x, c, t, ones)
v_u = unet(x, c, t, zeros)
x = x + h * (v_u + guidance_scale * (v_c - v_u))
```

Each row contains the digits 0 to 9, four samples per digit, same seed at every epoch.



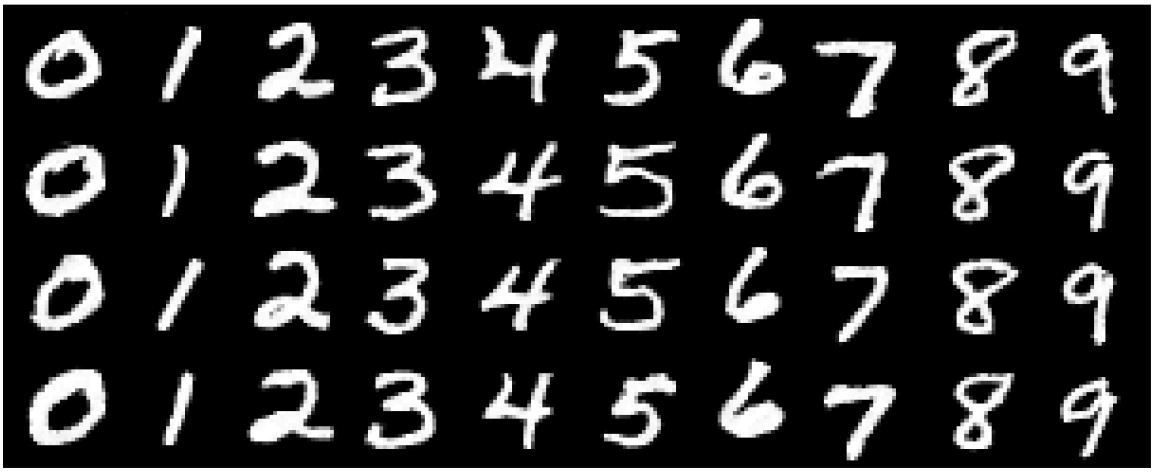
epoch 1

class_sched, epoch 5



epoch 5

class_sched, epoch 10

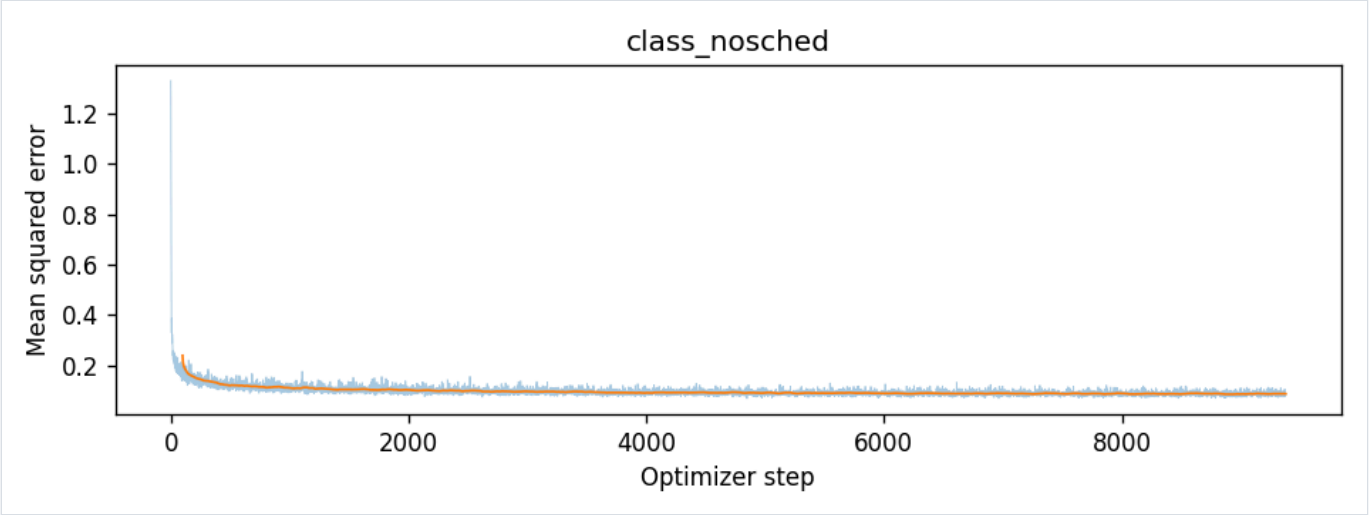


epoch 10

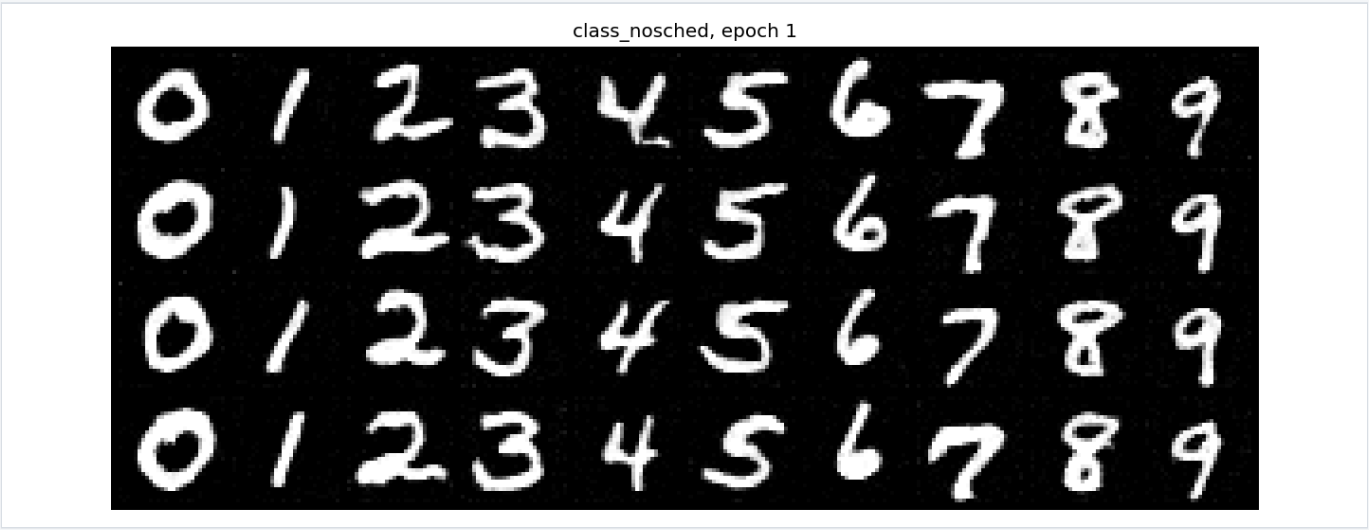
Class conditioning makes a large difference. Already after epoch 1, every column shows the requested digit, although the strokes are thick, jagged and slightly noisy. After epoch 5 the digits are clean and look like real handwriting, and epoch 10 is only slightly better. Compared to the time-only model, there are no more broken or mixed digits. The four samples of each digit still differ in slant, thickness and style, so the guidance does not collapse them into one template.

Removing the learning rate scheduler

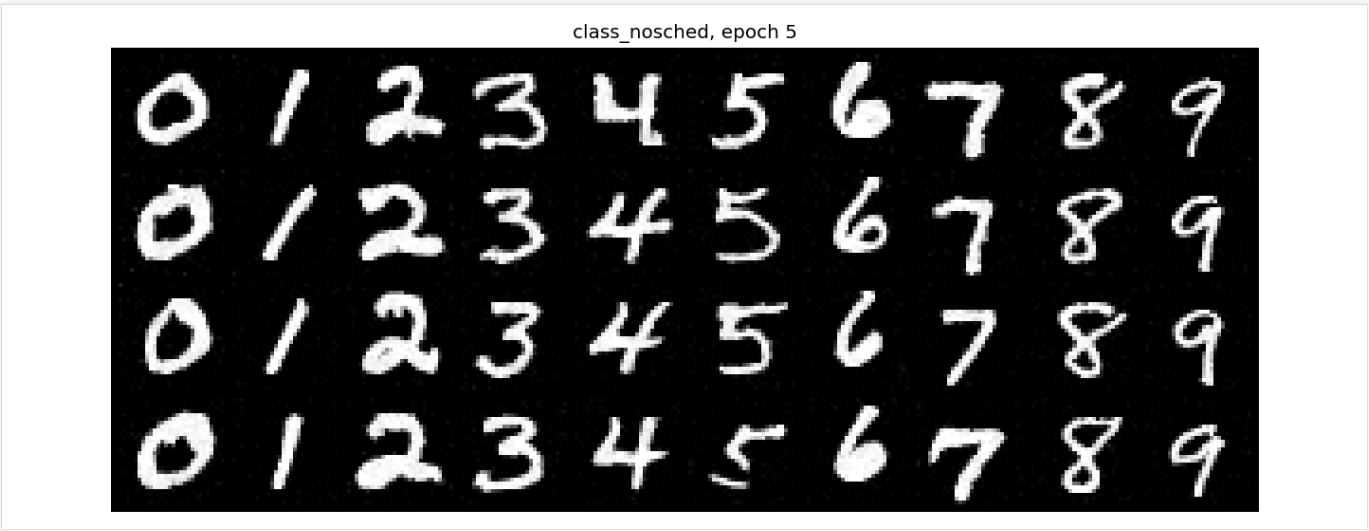
Without the scheduler, the learning rate stays at its initial value for the whole training. With 1e-2, the updates would stay large until the end, and the model would keep jumping around instead of settling. To compensate, I used a smaller constant learning rate of 1e-3, which is close to the learning rate the exponential schedule reaches in its last epoch (about 1.3e-3). Everything else is unchanged.



training loss without scheduler, constant lr = 1e-3

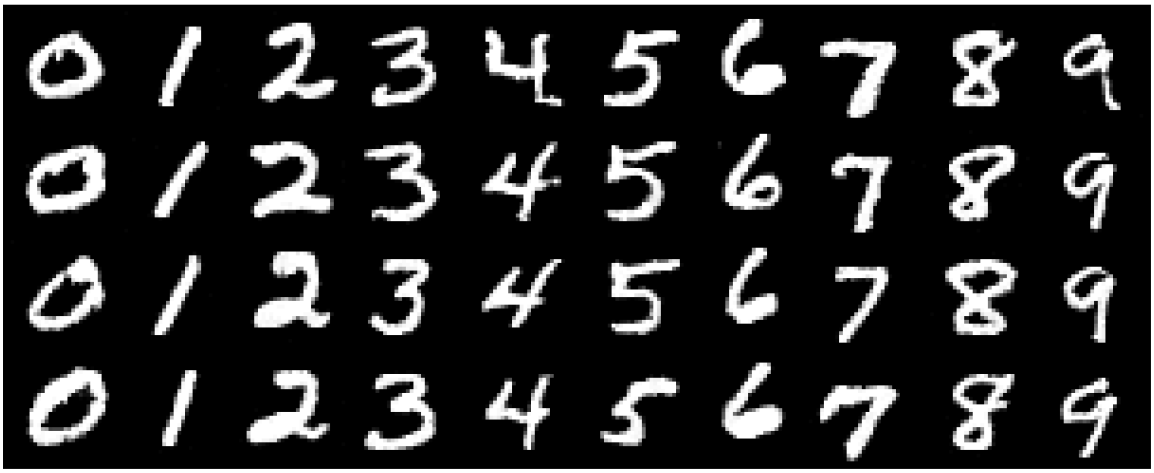


without scheduler, epoch 1



without scheduler, epoch 5

class_nosched, epoch 10



without scheduler, epoch 10

The constant learning rate works at least as well as the scheduler. The loss starts at about 1.3 instead of 12, since there is no large first update, and ends at the same level of about 0.09. The difference is most visible after epoch 1: the samples are already clean, while the model with the scheduler still produces thick and jagged digits at that point. After 10 epochs both models give digits of very similar quality, with only a few slightly irregular ones (for example the first 4) in the version without scheduler. The trade-off is that the constant learning rate cannot take the large steps at the beginning that make 1e-2 useful for longer trainings, but for ten epochs on MNIST the smaller rate is simpler and just as good.

What I learned

The most important thing I learned is why generation has to be iterative. A one-step denoiser trained on pure noise can only learn the average digit, but the same kind of network, asked many times for a small step in the right direction, produces sharp and varied images. Part A showed how much can be done with that sampling loop alone: editing, anagrams and hybrid images only change how the velocity is computed in each step.

All projects overview